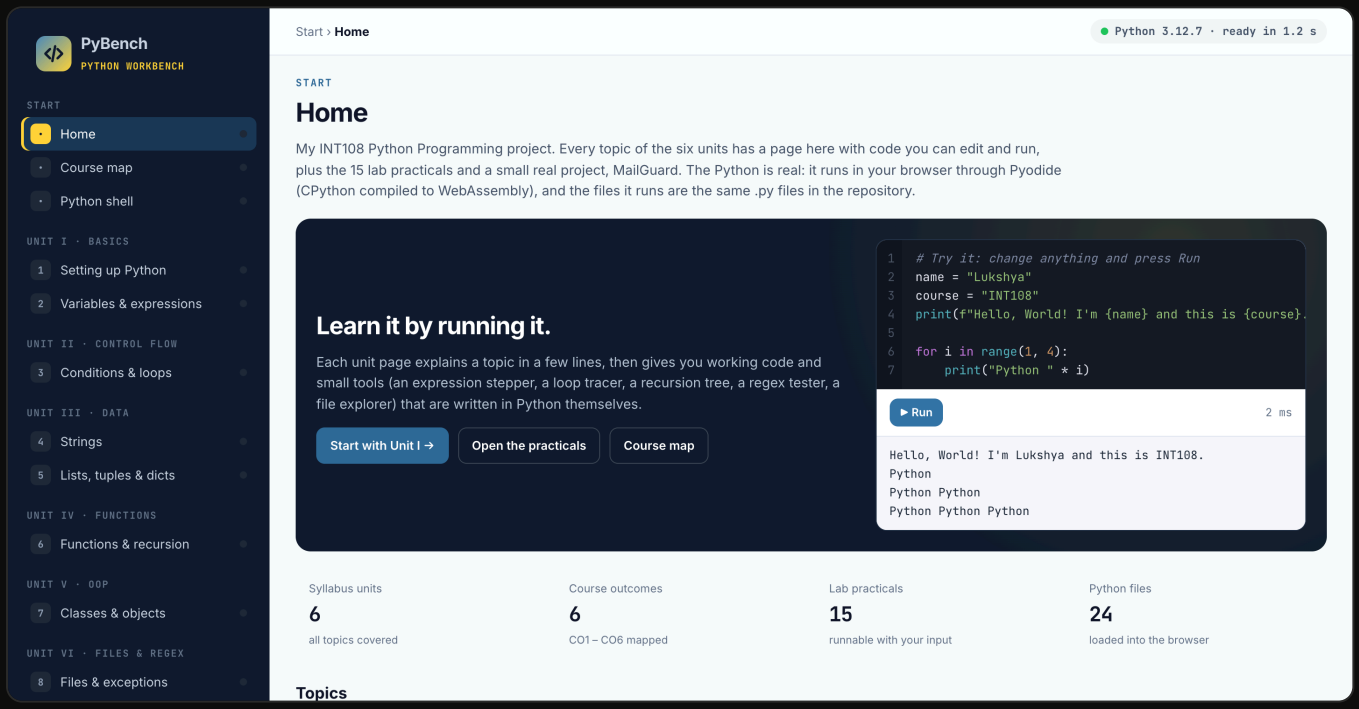
PyBench

A Python workbench that runs real Python in the browser. Every topic of the six INT108 units has a page with code you can edit and run, alongside small tools written in Python. It also has all fifteen lab practicals and MailGuard, a phishing mail analyser that brings every unit together.



STUDENT Lukshya Singh Supyal
REGISTRATION NO.
PROGRAMME B.Tech CSE, Semester 5
COURSE INT108 Python Programming (L:3 T:0 P:2, session 2026-27)
LIVE WEBSITE lushadowx.github.io/PyBench
SOURCE CODE github.com/LuShadowX/PyBench

CONTENTS

What's in this report

01 **Project overview**
Idea, sections, numbers

02 **How it's built**
Python in the browser, files

03 **Home, course map and shell**
Finding your way

04 **Unit I**
Environment, variables, expressions

05 **Unit II**
Conditions and loops

06 **Unit III**
Strings, lists, tuples, dictionaries

07 **Unit IV**
Functions and recursion

08 **Unit V**
Classes and OOP

09 **Unit VI**
Files, exceptions, regular expressions

10 **The 15 practicals**
Every program with a real run

11 **MailGuard**
The mini project and its notebook

12 **Syllabus and CO mapping**
All 73 topics, 6 COs

13 **Code walkthrough**
How the important parts work

14 **Testing**
What was checked and the results

15 **Deployment and conclusion**
GitHub Pages, what I learned

> I took every screenshot in this report from PyBench while using it. The outputs in them come from real runs of the Python code. Every code extract is copied from the source files, with the file name and line numbers. The practical outputs in chapter 10 come from running the files on my laptop.

Course and scope

The project follows the INT108 Python Programming syllabus for session 2026-27, the current version on the university notes site; I found no other code or version for this course. All six units, all six course outcomes and all fifteen experiments from the list of practicals are covered. Chapters 04 to 09 go through the syllabus unit by unit, chapter 10 has the practicals and chapter 12 maps every single topic to the page and tab that covers it.

Course outcomes

CO1
describe the installation of python environment and basics of Python language
Chapters 03, 04 · practical 1

CO2
apply the conditional and iterative statements for evaluating the appropriate alternates
Chapter 05 · practicals 2–5, 12

CO3
explore functions, including recursion, with parameters and arguments in Python
Chapter 07 · practical 7

CO4
construct the core data structures like lists, dictionaries, tuples and sets to store, process and sort the data
Chapter 06 · practicals 6, 13, 14

CO5
apply the concepts of object-oriented programming using encapsulation, polymorphism and inheritance
Chapters 08, 11 · practical 13

CO6
examine file handling operations and effectively apply regular expressions for pattern matching
Chapters 09, 11 · practicals 8–11, 14, 15

01 · PROJECT OVERVIEW

Learn Python by running it

Notes on Python usually show a program and its output side by side. PyBench lets you change the program and see the new output straight away. Every page is a short explanation followed by working code. Next to the code are small tools that show what Python is doing: an expression stepper for the order of operations, a loop tracer, a slice visualizer, a recursion tree, a regex tester and a file explorer. The tools are written in Python too, and the page calls them.

6 / 6
units, 73 syllabus topics mapped

15
lab practicals, runnable with your input

49
tabs on 15 pages, over 100 code cells

~5,600
lines: 2,367 Python, 3,250 JS / HTML / CSS

Sections

SECTION	PAGES AND TABS	SYLLABUS
Unit I · Basics	Setting up Python (versions, Windows, Hello World, tools) · Variables & expressions (names, types, operators and order, strings, comments)	Unit I · CO1
Unit II · Control flow	Conditions & loops: modulus, Boolean logic, conditionals, random numbers, while and for, loop tracer, encapsulation and generalization	Unit II · CO2
Unit III · Data	Strings (5 tabs) · Lists, tuples & dicts (lists, nested lists, tuples, dictionaries, sparse matrices, aliasing, sets and sorting)	Unit III · CO4
Unit IV · Functions	Calls and conversion, math, adding new functions, parameters and arguments, recursion, uses of recursion	Unit IV · CO3
Unit V · OOP	Classes and objects, attributes, inheritance and overriding, data hiding, overloading	Unit V · CO5
Unit VI · Files & regex	Files & exceptions (text files, variables, directories, pickling, exceptions, file explorer) · Regular expressions (concept, tester, match, types, web scraping)	Unit VI · CO6
Lab	The 15 practicals with aim, code, input, output and viva questions · Lab record · MailGuard project	List of practicals · all COs
Start	Home · Course map · Python shell (REPL)	CO1

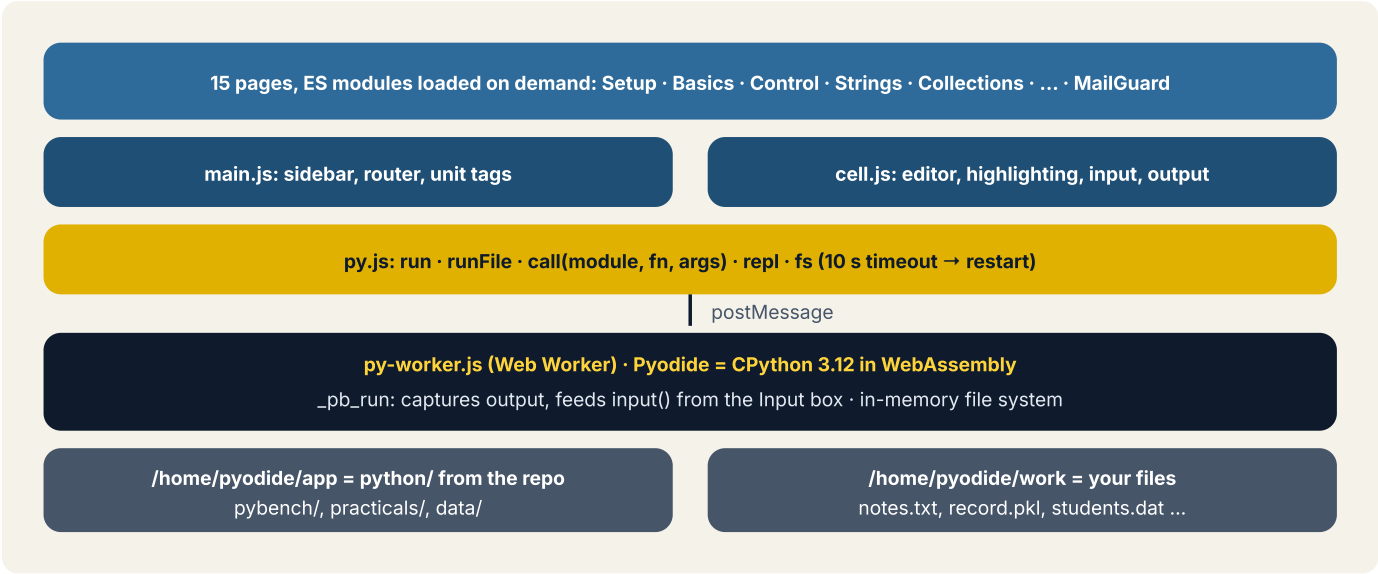
What makes it more than a set of notes

- **It is real Python.** The site runs CPython 3.12 compiled to WebAssembly (Pyodide). The `.py` files in the repository are copied into the browser's Python when it starts, so the practicals and tools run exactly the code that is in the project.
- **The same code runs on a computer.** `python -m pybench` opens a menu with the unit demos, the practicals and MailGuard. `python -m unittest` runs 46 tests, and everything uses only the standard library.
- **The tools explain Python with Python.** The expression stepper parses your expression with the `ast` module. The loop tracer records every executed line with `sys.settrace`. The recursion tree is built by a recursive function.
- **A small real project.** MailGuard grew out of practical 14. It reads mails from files, applies six rule classes and explains its score, and it has a Kaggle / Jupyter notebook version with tables and charts.

02 · HOW IT'S BUILT

Python in the browser

The site is plain HTML, CSS and JavaScript modules with no framework and no build step, hosted on GitHub Pages. Python runs in a Web Worker, so a long or endless loop never freezes the page. The page sends code to the worker, and the worker sends back what it printed.



Files

FILE	JOB	LINES
`python/pybench/unit1_basics.py`	environment info, name checker, NameError helper, type inspector, expression stepper, string operations, comment counter	168
`python/pybench/unit2_control.py`	modulus facts, clock, truth tables, grade / ticket / leap year, dice, guess game, Monte Carlo π, patterns, primes, loop tracer	219
`python/pybench/unit3_data.py`	traversal, slices, find, counting, comparison, list demos, matrix, tuples, word count, SparseMatrix, aliasing, sets, sorting	214
`python/pybench/unit4_functions.py`	conversion, coercion, math table, parameters, factorial, fib, memo, fast power, Hanoi, recursion tree	169
`python/pybench/unit5_oop.py`	Student, BankAccount, Shape hierarchy, Vector, Calculator	204
`python/pybench/unit6_files.py`	text files, variables, directories, pickle, exceptions, regex patterns and reports, validation, scraping	192
`python/pybench/mailguard.py`	Mail, Rule and six rule subclasses, Finding, Report, MailGuard	258
`python/practicals/p01 … p15`	the fifteen lab programs, each with testable functions and a `main()`	15 files
`python/tests/`	unittest tests for the units, the practicals and MailGuard	46 tests
`js/core/py-worker.js`	loads Pyodide, copies the Python files in, runs code with captured output and fed input, REPL, file access	155
`js/core/py.js`	promise API for the pages, timeouts that restart the worker, Python status	92

FILE	JOB	LINES
`js/core/cell.js`	Python syntax highlighter, editor (highlighted layer under a textarea), runnable cell	132
`js/pages/*.js`	one module per page, 15 in total	2,054
`notebooks/mailguard_analysis.ipynb`	MailGuard with pandas tables and matplotlib charts	19 cells

03 · HOME, COURSE MAP AND SHELL

Finding your way

The sidebar groups the pages by syllabus unit and puts a green dot next to every page you have opened. The top bar shows which unit, course outcome and practical each page covers, plus a status pill for Python. Python starts loading as soon as the site opens and is usually ready in one to two seconds once cached.

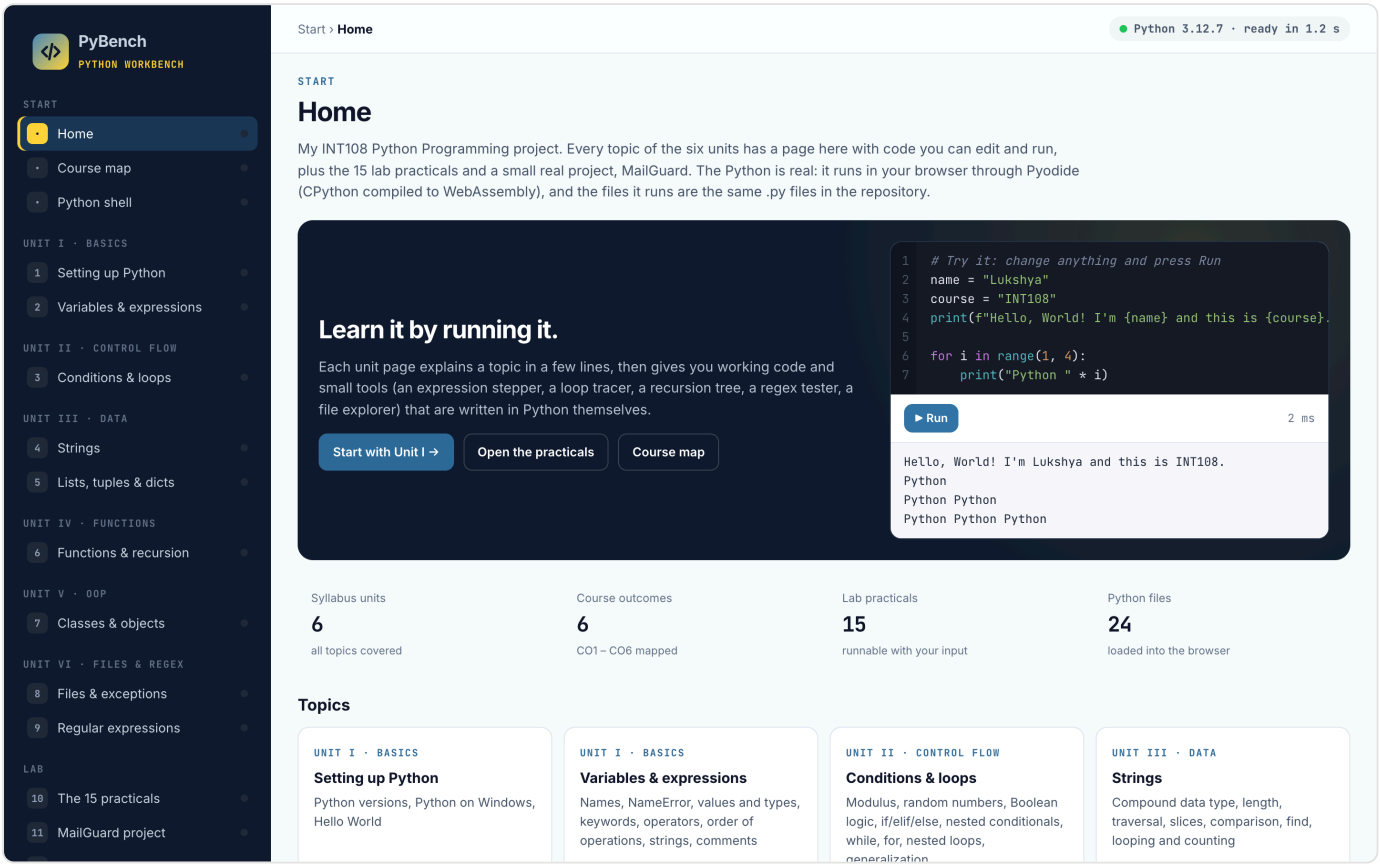


Fig. 1 · **Home.** The hero cell is live: it has been run here. Below it are the topic cards, the six course outcomes with links, and a short note on how the site runs Python.



Fig. 2 · **Python shell.** The interactive interpreter (CO1). Expressions are echoed, a `for` block continues on "..." lines, variables persist between lines, and errors show a normal traceback.

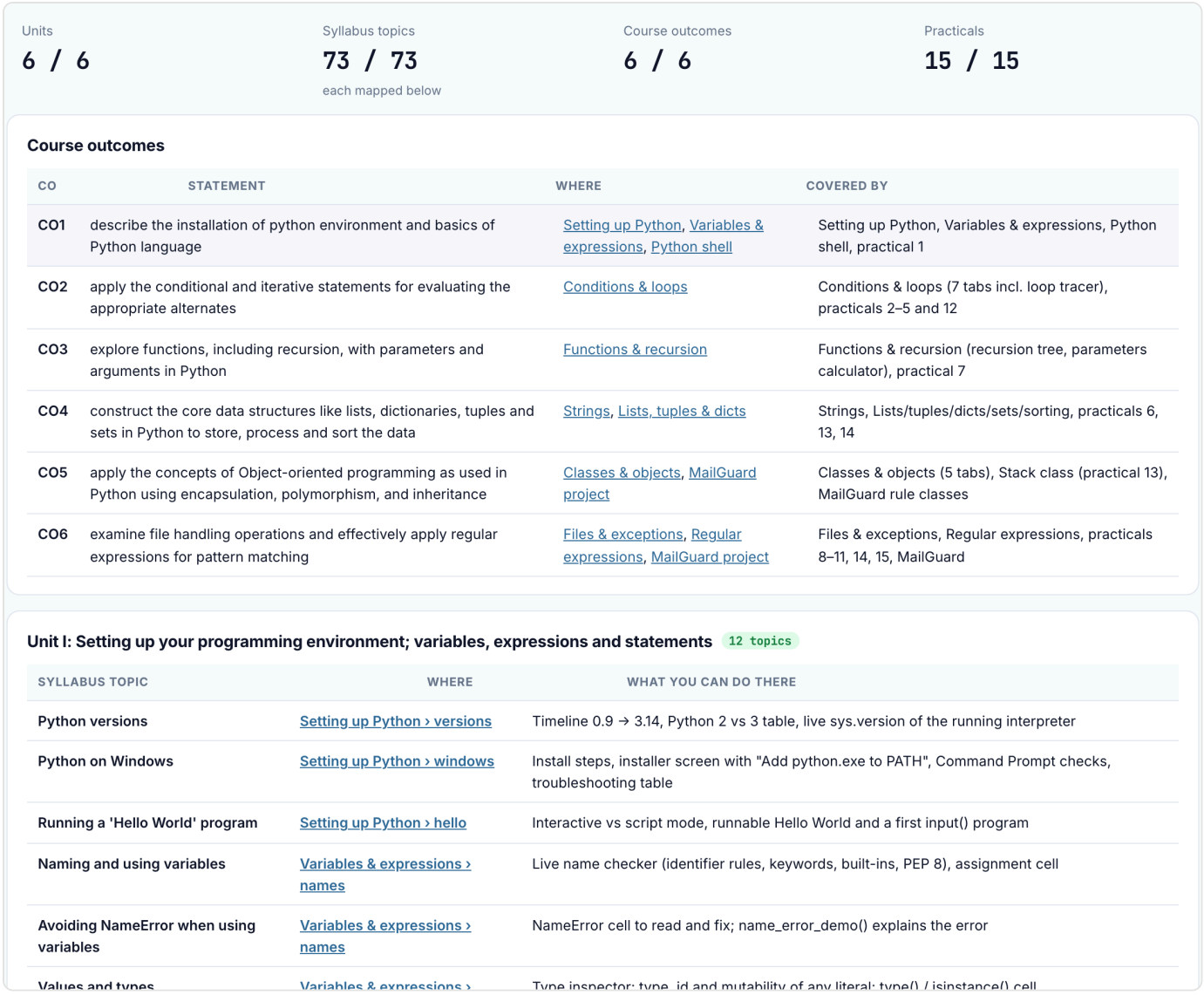


Fig. 3 · **Course map.** Units, topics, COs and practicals counted, the CO table with links, and the first unit's topics, each linked to its page and tab. The full mapping is in chapter 12.

04 · UNIT I

Setting up Python, variables and expressions

Unit I: Python versions, Python on Windows, running a "Hello World" program. Then naming and using variables, avoiding NameError, values and types, variable names and keywords, statements, operators and operands, order of operations, operations on strings, composition and comments. `C01` `Practical 1`

Timeline

YEAR	VERSION	WHAT CHANGED
1991	0.9	Guido van Rossum releases the first version (classes, exceptions, functions, lists, dicts already there)
1994	1.0	lambda, map, filter and reduce
2000	2.0	list comprehensions, garbage collector, Unicode strings
2008	3.0	the big cleanup: print() becomes a function, / is true division, str is Unicode. Not backward compatible
2010	2.7	last Python 2 release
2016	3.6	f-strings: f"{name}"
2020	2.7 EOL	Python 2 reaches end of life on 1 January 2020, no more fixes
2021	3.10	match / case pattern matching, better error messages
2023	3.12	faster interpreter, f-string improvements (the version PyBench runs in the browser)
2024	3.13	new interactive shell with colours, experimental free-threading
2025	3.14	template strings, deferred annotations (what I use on my laptop)

The Python running this page

`version`	`3.12.7`
`implementation`	`CPython`
`platform`	`emscripten`
`executable`	`/home/pyodide/this.program`
`keywords`	`35`

This comes from `pybench.unit1_basics.environment()`, which reads `sys.version_info` and `platform`.

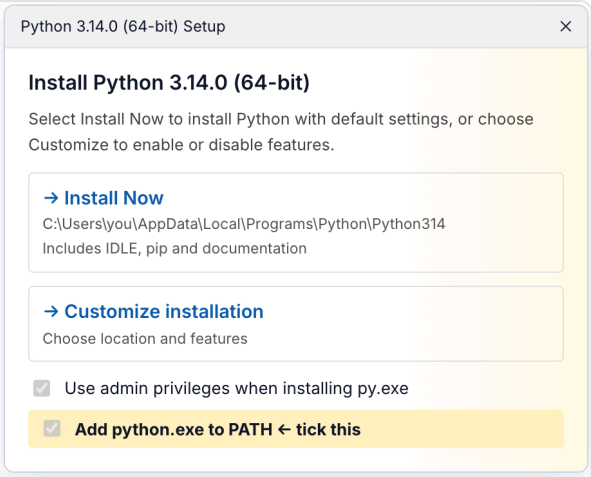
Python 2 vs Python 3

	PYTHON 2	PYTHON 3
print	print "hi"	print("hi")
division	7 / 2 == 3	7 / 2 == 3.5 and 7 // 2 == 3
input	raw_input() for text, input() evaluates	input() always returns a str
strings	bytes by default, u"..." for Unicode	Unicode by default, b"..." for bytes
range	range() builds a list, xrange() is lazy	range() is lazy
integers	int and long	one int type with unlimited size

Fig. 4 · **Python versions.** The timeline from 0.9 to 3.14, the Python 2 vs 3 differences, and the Python the page is running on, read with `sys.version_info` by `environment()`.

Steps

1. **Download** – Open python.org → Downloads → Windows and download the latest Python 3 "Windows installer (64-bit)". The Microsoft Store version also works for learning.
2. **Tick "Add python.exe to PATH"** – This is the step everyone misses. Without it Command Prompt says "python is not recognized as an internal or external command".
3. **Install Now** – Installs Python, pip, IDLE and the py launcher for the current user. "Customize installation" lets you pick the folder or install for all users.
4. **Disable path length limit** – The last screen offers this. Click it so long folder paths don't break pip installs.
5. **Check it** – Open Command Prompt (Win + R, cmd) and run the commands below.



Check the install (Command Prompt)

```
C:\> python --version
Python 3.14.0

C:\> py -0          # every installed version
 -V:3.14 *        Python 3.14 (64-bit)

C:\> pip --version
pip 25.2 from ...\Python314\Lib\site-packages\pip (python

C:\> python hello.py
Hello, World!
```

If something goes wrong

PROBLEM	FIX
"python is not recognized"	PATH was not ticked. Re-run the installer → Modify → Add Python to environment variables, or use py instead of python
python opens the Microsoft Store	Settings → Apps → Advanced app settings → App execution aliases → turn off the python.exe alias
pip is not recognized	Use python -m pip install <package>
Two versions clash	py -3.14 file.py picks a version explicitly

Fig. 5 · **Python on Windows.** The install steps, the installer screen with "Add python.exe to PATH" highlighted, the checks to run in Command Prompt, and fixes for the usual problems.

Interactive mode

```
C:\> python
Python 3.14.0 (tags/v3.14.0) on win32
Type "help", "copyright", "credits" or "license" for more
>>> print("Hello, World!")
Hello, World!
>>> 2 + 3
5
>>> exit()
```

In interactive mode the value of an expression is echoed automatically. Try it on the Python shell page.

Script mode

```
# hello.py
print("Hello, World!")

C:\> python hello.py
Hello, World!
```

In a script nothing is echoed, you have to print() what you want to see.

Fig. 6 · **Interactive mode and script mode.** In interactive mode the value of each expression is echoed; in a script only what you print() is shown.

A slightly bigger first program

input() reads one line from the Input box below the code.

```
name = input("What is your name? ")
year = int(input("Which year were you born? "))
print("Hello,", name + "!")
print("You will turn", 2026 - year, "this year.")
```

Input (one line per input() call)

```
Lukshya
2005
```

▶ Run | Reset | 1 ms

```
What is your name? Lukshya
Which year were you born? 2005
Hello, Lukshya!
You will turn 21 this year.
```

Fig. 7 · **A first program with input().** Both input() calls read from the Input box, which also shows what was typed.

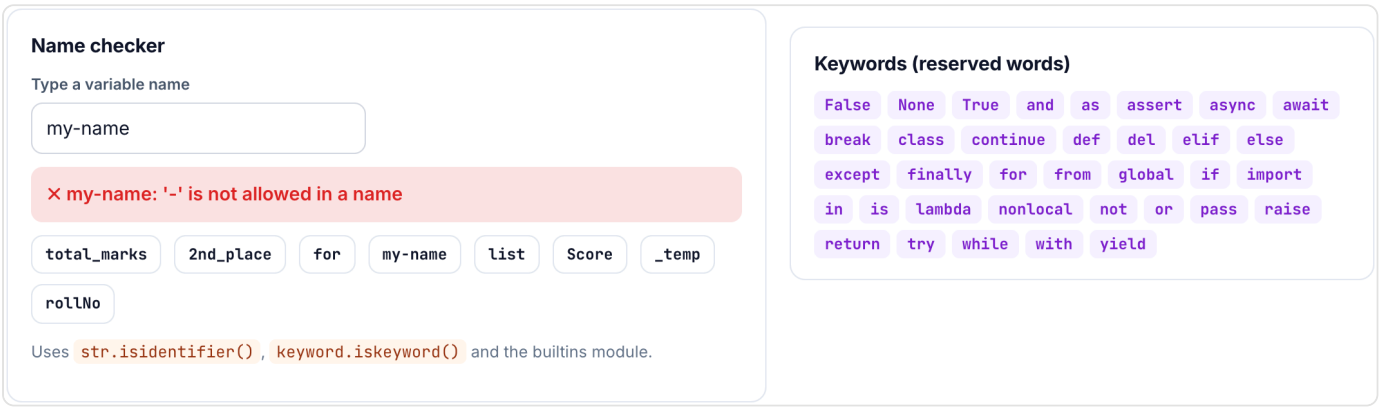


Fig. 8 · **Naming and using variables.** The name checker explains why `my-name` is not allowed. All 35 keywords are listed straight from `keyword.kwlist`.

Avoiding NameError

Run it, read the error, then fix the spelling.

```
score = 90
print(score)
print(Score)   # Python is case sensitive
```

▶ Run | Reset | 3 ms

```
90
Traceback (most recent call last):
  File "<cell>", line 3, in <module>
NameError: name 'Score' is not defined. Did you mean: 'score'?
```

Fig. 9 · **Avoiding NameError.** Python 3.12 even suggests the right name: "Did you mean: 'score'?"

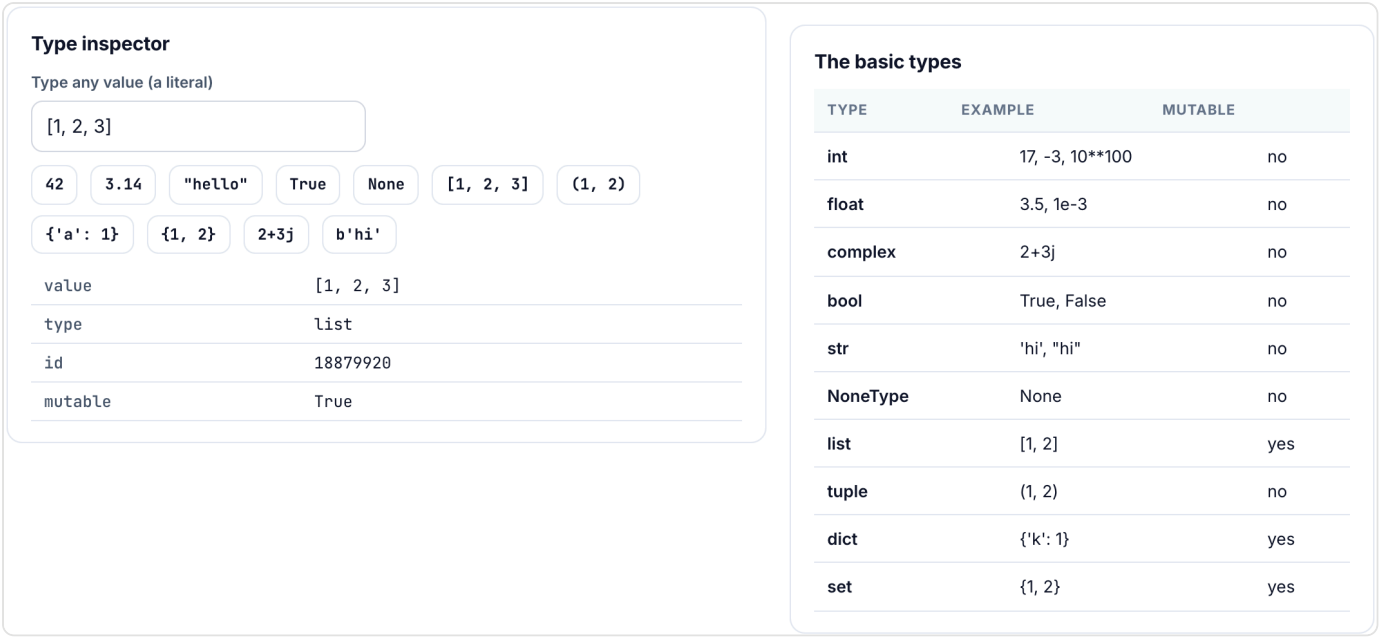


Fig. 10 · **Values and types.** The type inspector shows the type, identity and mutability of any literal, here a list, next to the table of basic types.

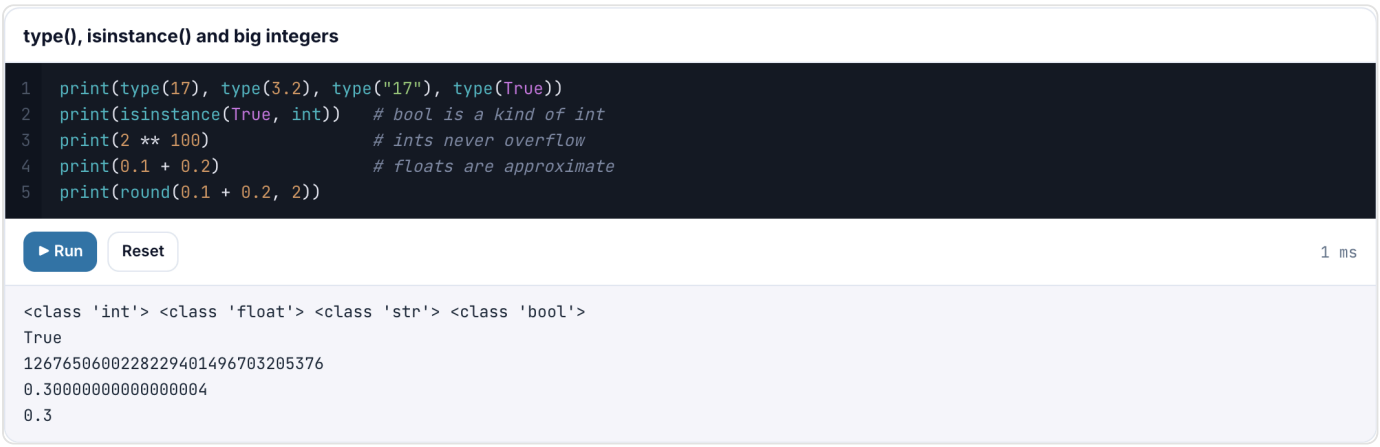


Fig. 11 · **type(), isinstance() and big integers.** bool is a kind of int, ints never overflow, and floats are approximate.



Fig. 12 · **Order of operations.** The expression stepper takes `2 + 3 * 4 ** 2 // 5 - 1`: ** first, then * and //, then + and −, left to right. The buttons try `2 ** 3 ** 2` (right to left) and `0 and 1 / 0` (short-circuit).

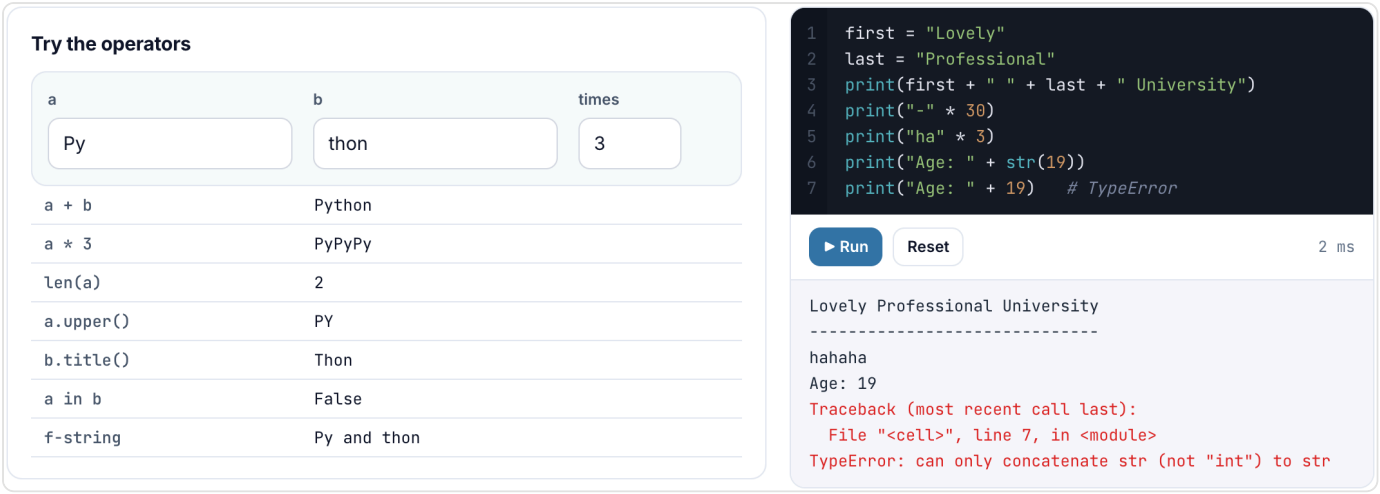


Fig. 13 · **Operations on strings.** + and * with live inputs, and the TypeError you get for "Age: " + 19.

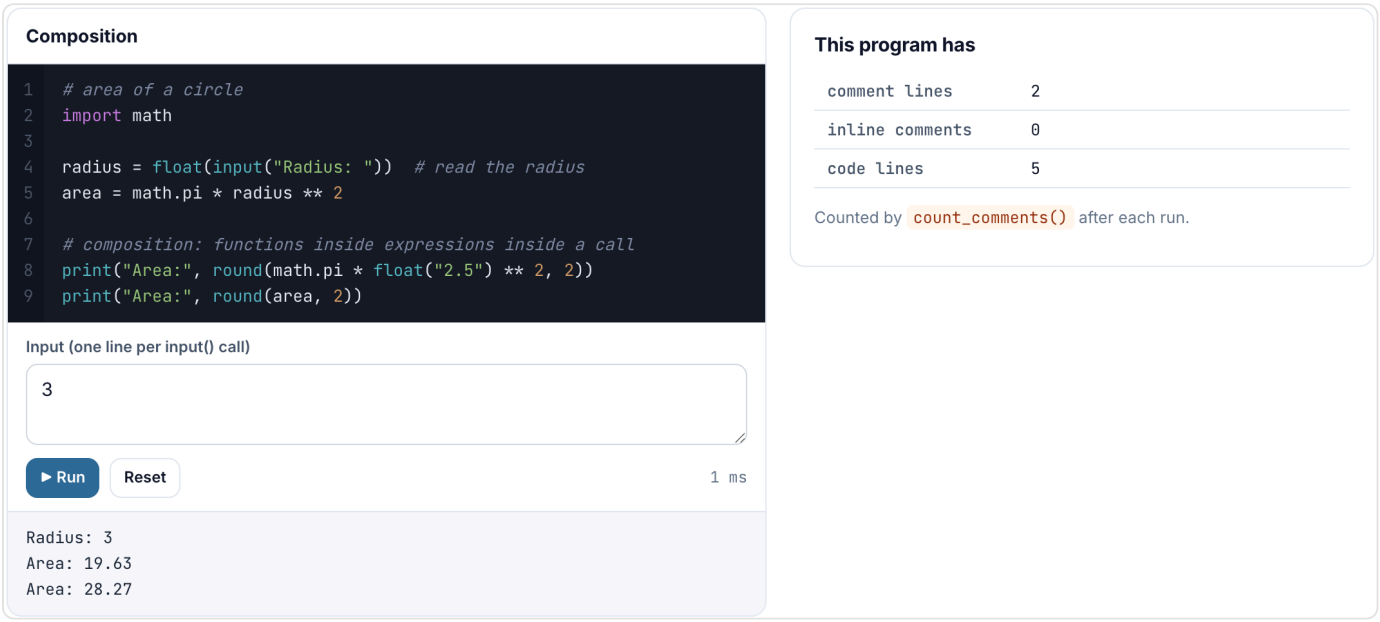


Fig. 14 · **Composition and comments.** A program built by composition, run with input 3, and its comment lines counted by `count_comments()`.

05 · UNIT II

Conditional and iterative statements

Unit II: modulus operator, random numbers, Boolean expressions, logic operators, conditional and nested conditionals. Then while statements, for loop statement, nested for, nested while, random numbers in loops, encapsulation and generalization. CO2

Practicals 2-5, 12

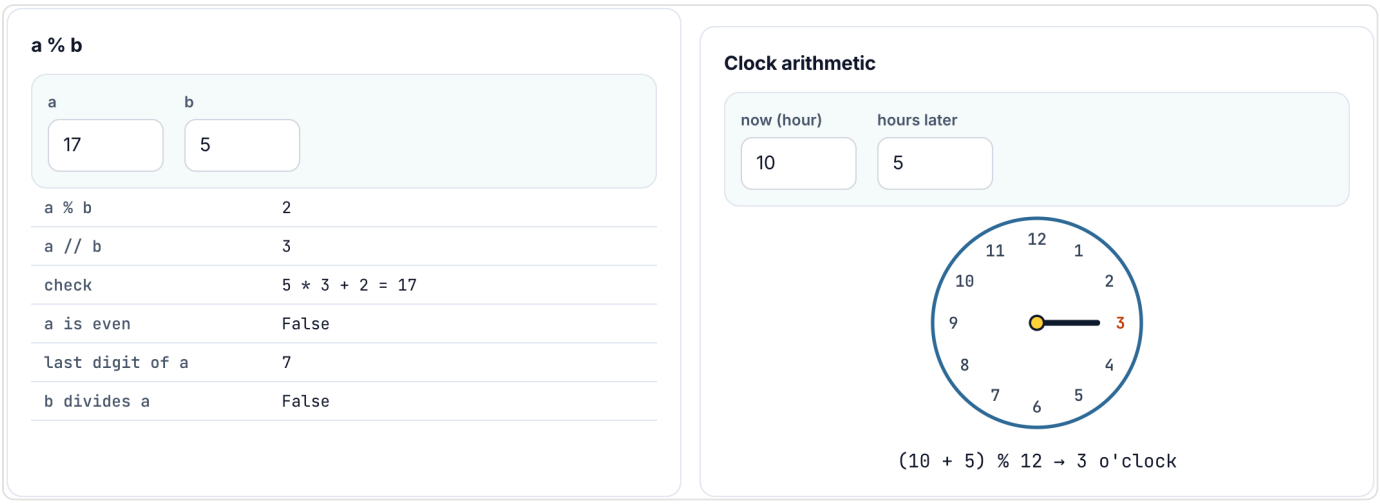


Fig. 15 · **Modulus operator.** Everything % tells you about 17 and 5, and clock arithmetic: (10 + 5) % 12 is drawn as 3 o'clock.

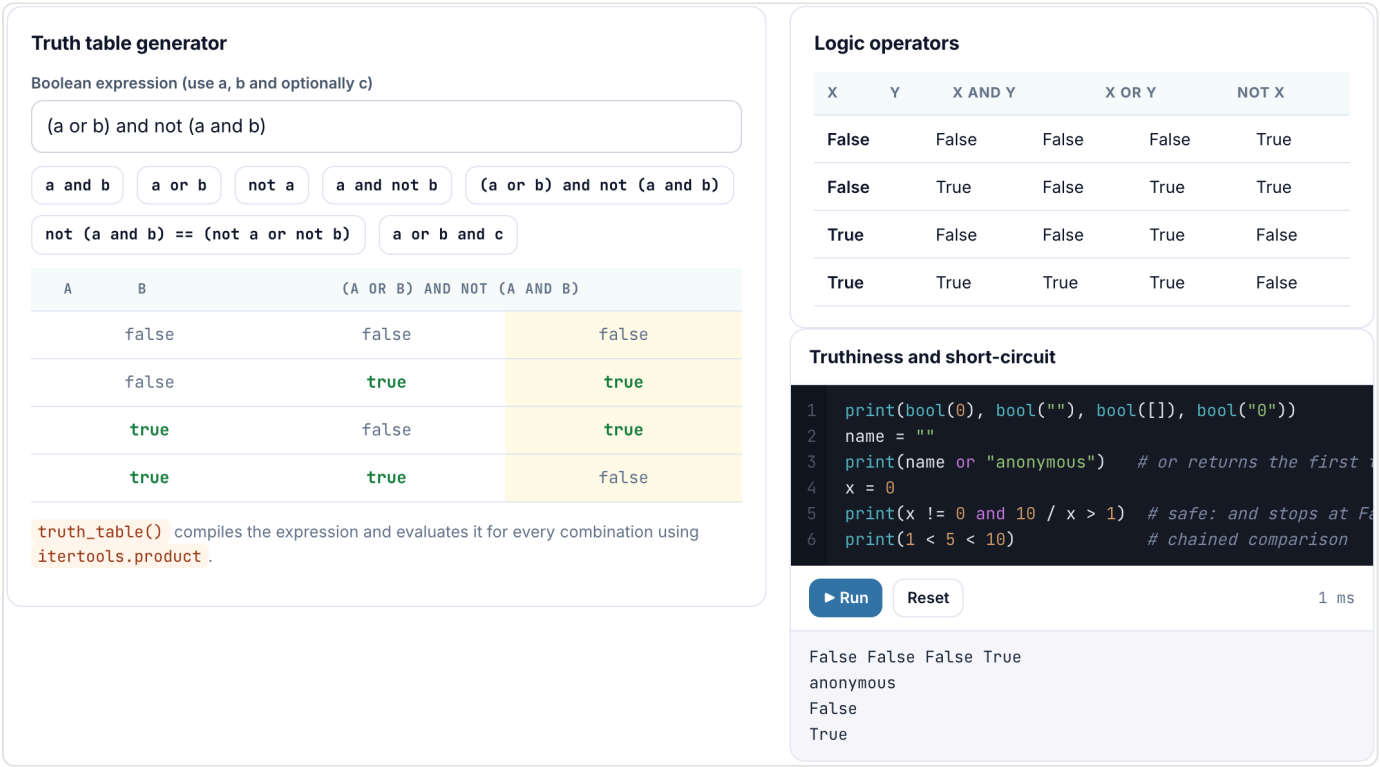


Fig. 16 · **Boolean expressions and logic operators.** The truth table generated for `(a or b) and not (a and b)`, which turns out to be XOR, next to the and / or / not table and short-circuit examples.

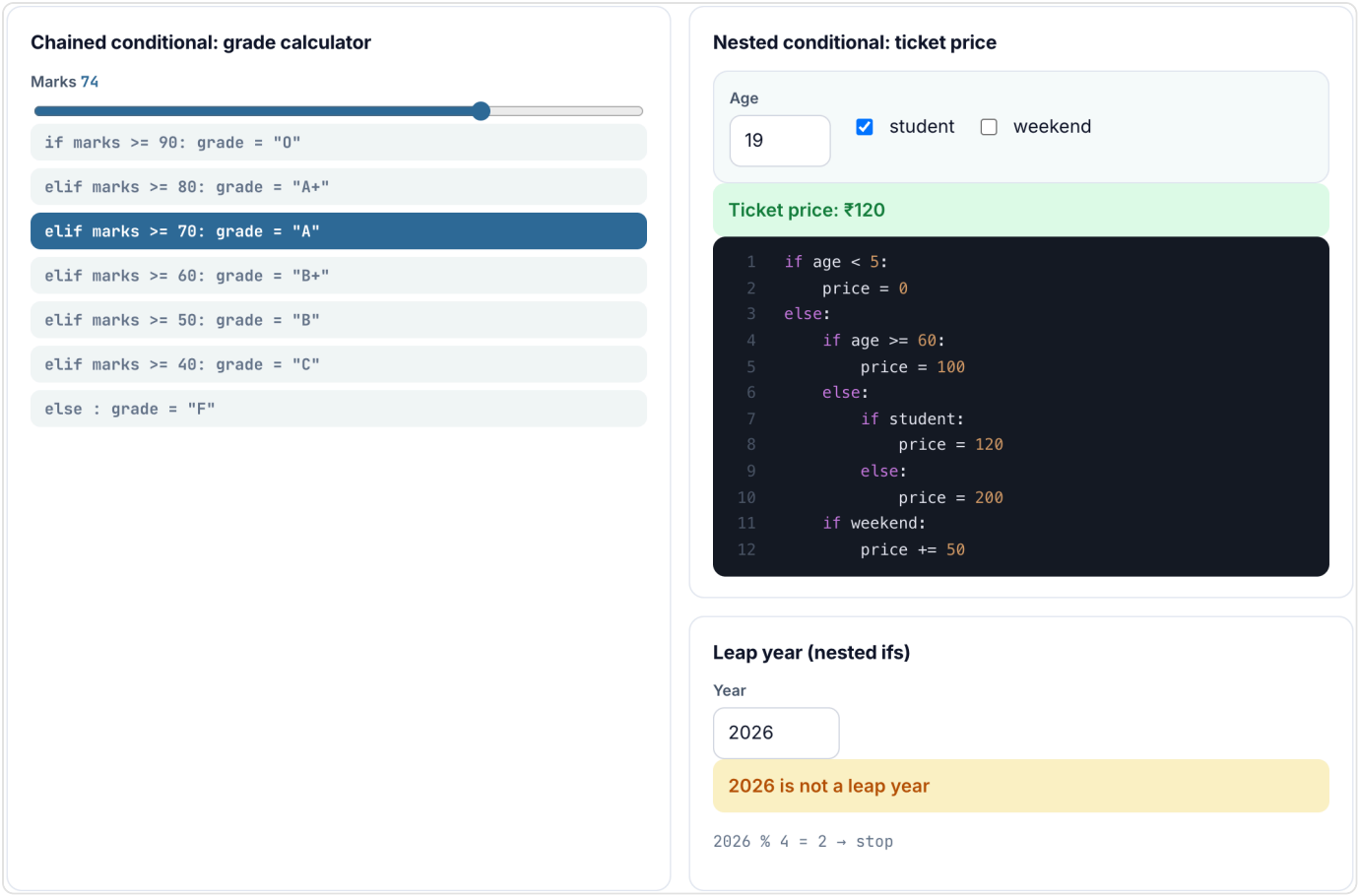


Fig. 17 · **Conditional and nested conditionals.** For 74 marks the grade ladder highlights the `elif marks >= 70` branch. Next to it: the nested ticket-price conditional and a leap-year check with its % steps.



Fig. 18 · **Random numbers.** 600 dice rolls with seed 42: each face comes up between 84 and 107 times against an expected 100, and the average roll is 3.477. On the right, π estimated from random points.

Nested for

```
for i in range(1, 4):
    for j in range(1, 4):
        print(f"{i}x{j}={i*j:<2}", end="  ")
    print()
```

▶ Run Reset 2 ms

```
1x1=1   1x2=2   1x3=3
2x1=2   2x2=4   2x3=6
3x1=3   3x2=6   3x3=9
```

Fig. 19 · **Nested for.** A multiplication grid.

Nested while (primes)

```
n = 2
while n <= 30:
    d = 2
    while d * d <= n and n % d != 0:
        d += 1
    if d * d > n:
        print(n, end=" ")
    n += 1
```

▶ Run Reset 1 ms

```
2 3 5 7 11 13 17 19 23 29
```

Fig. 20 · **Nested while.** The inner loop tries divisors to find primes.

break, continue and loop else

```
for n in range(1, 20):
    if n % 2 == 0:
        continue          # skip even numbers
    if n > 9:
        break             # stop the loop
    print(n, end=" ")
print()

for d in range(2, 7):
    if 49 % d == 0:
        print("49 divisible by", d)
        break
else:
    print("no divisor between 2 and 6")
```

▶ Run Reset 2 ms

```
1 3 5 7 9
no divisor between 2 and 6
```

Fig. 21 · **break, continue and loop else.** The else part runs only when the loop wasn't broken.

Step 3 · generalize: n becomes a parameter

```
def print_multiples(n):
    i = 1
    while i <= 6:
        print(n * i, end="\t")
        i += 1
    print()

print_multiples(3)
print_multiples(4)
```

▶ Run Reset 1 ms

```
3       6       9       12      15      18
4       8       12      16      20      24
```

Fig. 22 · **Generalization, step 3.** n becomes a parameter of print_multiples.

while statement

```
# Collatz: halve if even, 3n + 1 if odd, until we reach 1
n = 27
steps = 0
while n != 1:
    n = n // 2 if n % 2 == 0 else 3 * n + 1
    steps += 1
print("reached 1 after", steps, "steps")
```

▶ Run Reset 1 ms

```
reached 1 after 111 steps
```

Fig. 23 · **while statement.** The Collatz sequence from 27 reaches 1 after 111 steps.

Step 4 · reuse it for a whole table, generalize the size too

```
def print_multiples(n, high):
    for i in range(1, high + 1):
        print(n * i, end="\t")
    print()

def print_mult_table(high):
    for n in range(1, high + 1):
        print_multiples(n, high)

print_mult_table(5)
```

▶ Run Reset 2 ms

```
1       2       3       4       5
2       4       6       8       10
3       6       9       12      15
4       8       12      16      20
5       10      15      20      25
```

Fig. 24 · **Generalization, step 4.** print_multiples reused for a whole table.

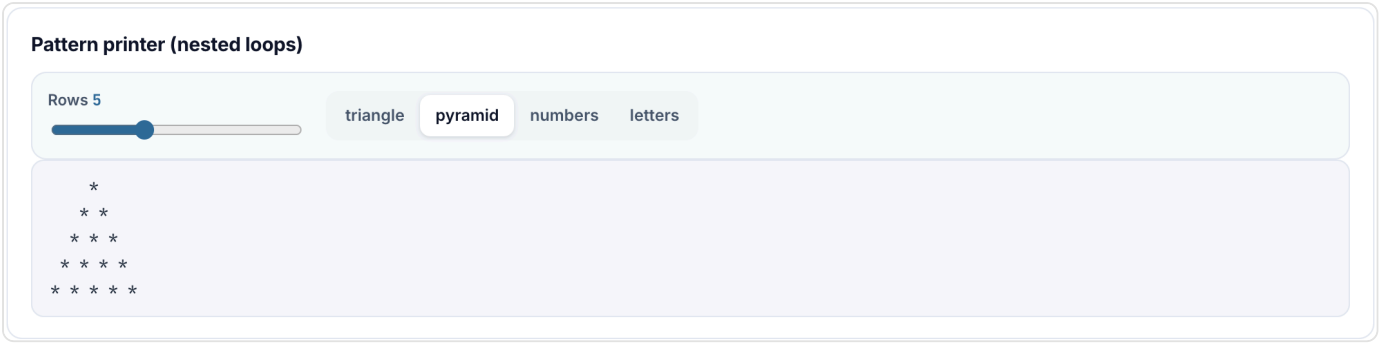


Fig. 25 · **Pattern printer.** Nested loops print a pyramid, a triangle, numbers or letters.

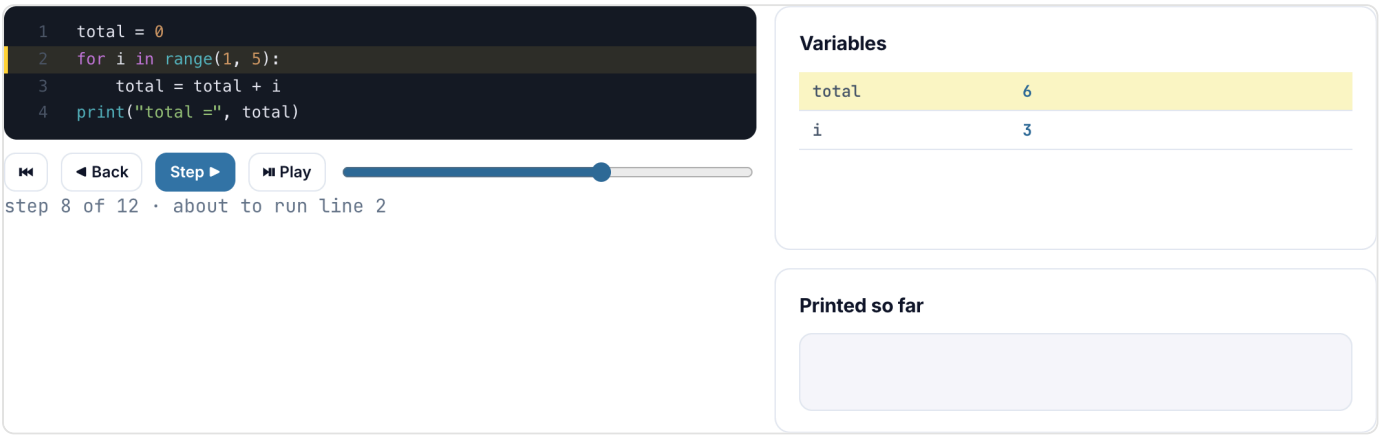


Fig. 26 · **Loop tracer.** Step 8 of the for loop: line 2 is about to run, `total` has just changed to 6 (highlighted) and `i` is 3. The steps are recorded by `sys.settrace`.

Strings, lists, tuples and dictionaries

Unit III: a string is a compound data type; length, traversal, slices, comparison, the find function, looping and counting. Lists: values, length, membership, operations, slices, deletion, accessing elements, lists and for loops, list parameters and nested lists. Tuples and dictionaries: mutability, tuple assignment, tuples as return values, dictionary operations and methods, sparse matrices, aliasing and copying. CO4 adds sets and sorting. `CO4` `Practicals 6, 13, 14`



Fig. 27 · **Slice visualizer.** `s[::-2]` on "LOVELY PROFESSIONAL": the characters it picks are raised in yellow, with positive and negative indexes, and the visited indices come from `slice_info()`.



Fig. 28 · **A compound data type and its length.** Indexing, len() and the IndexError past the end.

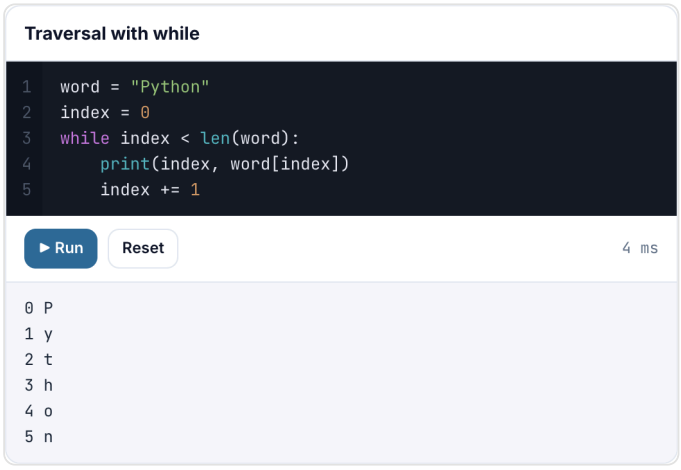


Fig. 29 · **String traversal** with a while loop and an index.

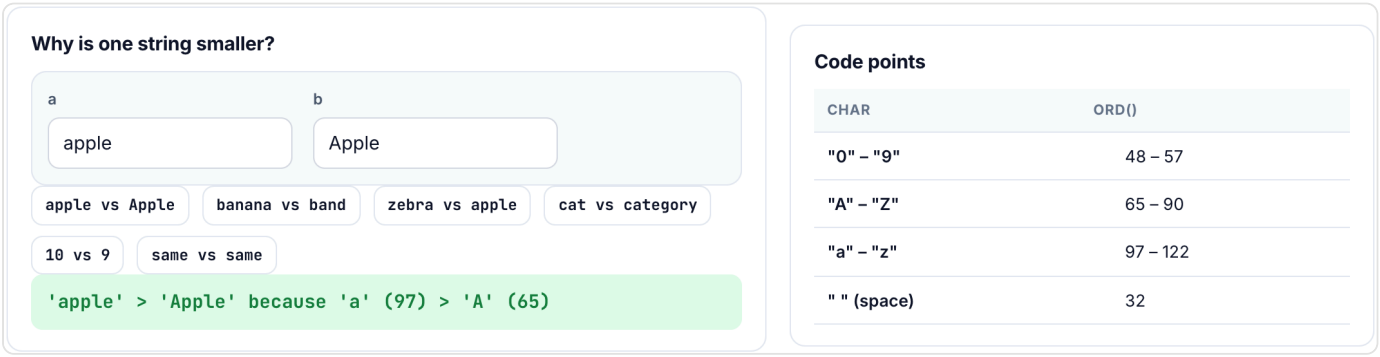


Fig. 30 · **Comparison.** "apple" > "Apple" because 'a' is 97 and 'A' is 65.

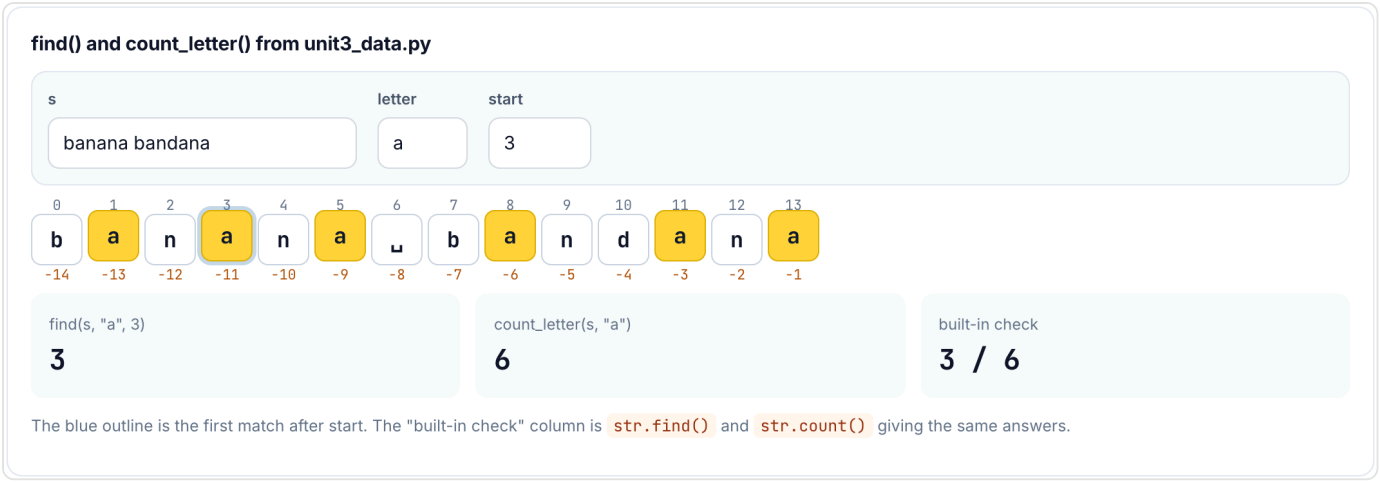


Fig. 31 · **The find function, looping and counting.** The hand-written `find()` searching "banana bandana" for "a" from index 3 (blue outline), every "a" highlighted, and the count, checked against the built-in methods.

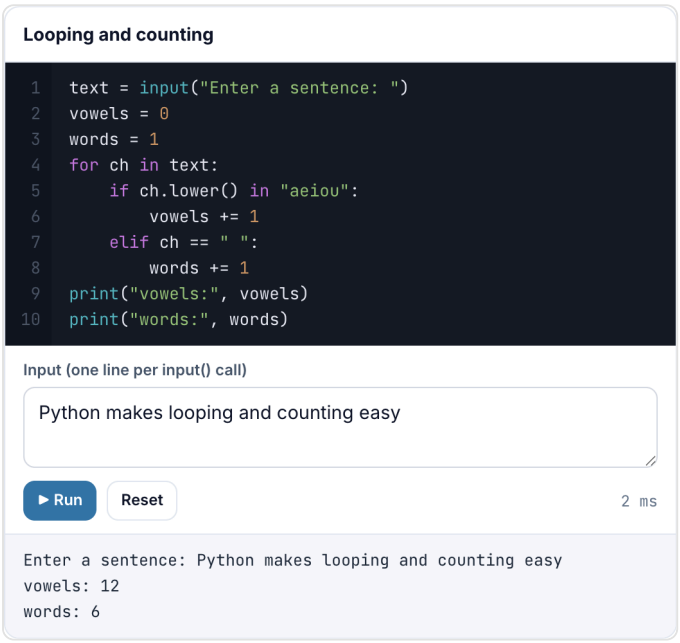


Fig. 32 · **Looping and counting.** Vowels and words counted with the counter pattern.

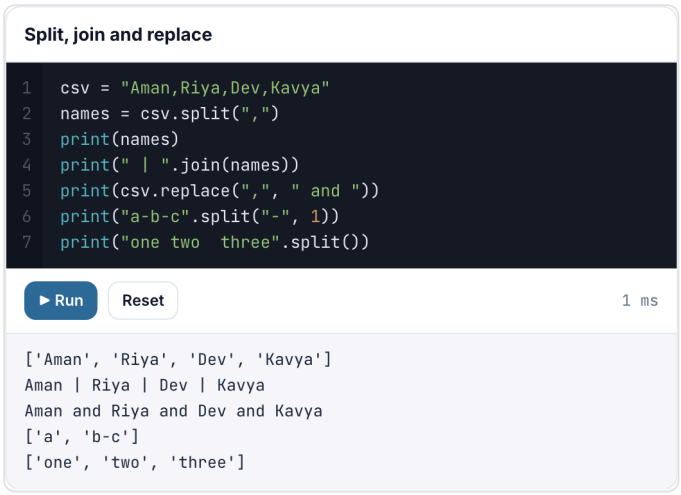


Fig. 33 · **String methods.** split, join and replace.



Fig. 34 · **Lists.** `list_demo()` on your own values (length, membership, slices, +, *, del) and a cell that changes a list in place.

List parameters

```python
def double_all(items):
    for i in range(len(items)):
        items[i] *= 2          # changes the caller's list

def doubled(items):
    return [x * 2 for x in items]   # builds a new list

nums = [1, 2, 3]
new = doubled(nums)
print(nums, new)
double_all(nums)
print(nums)
```

▶ Run | Reset | 1 ms

```
[1, 2, 3] [2, 4, 6]
[2, 4, 6]
```

Fig. 35 · **List parameters.** `double_all()` changes the caller's list; `doubled()` returns a new one.

Deletion

```python
nums = [10, 20, 30, 40, 50, 60]
del nums[0]
print(nums)
del nums[1:3]
print(nums)
nums.remove(60)
print(nums)
nums.clear()
print(nums, len(nums))
```

▶ Run | Reset | 1 ms

```
[20, 30, 40, 50, 60]
[20, 50, 60]
[20, 50]
[] 0
```

Fig. 36 · **Deletion.** del by index and by slice, remove() and clear().

Matrix as a nested list

rows: 3 cols: 4

m = matrix(rows, cols)

1	2	3	4
5	6	7	8
9	10	11	12

transpose(m)

1	5	9
2	6	10
3	7	11
4	8	12

```
[[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]
```

Fig. 37 · **Nested lists.** A 3×4 matrix built as a list of rows, and its transpose.

Tuple as return value

Numbers: 72, 85, 90, 66

```
low, high, avg = min_max_avg([72, 85, 90, 66])
```

low	66
high	90
avg	78.25

Mutability

From `tuple_is_immutable()`:

```
point[0] = 10 → TypeError: 'tuple' object does not support item assignment
```

Fig. 38 · **Tuples.** Three values returned as one tuple, and the TypeError from assigning to a tuple item.

Creating and unpacking

```
point = (3, 4)
single = (5,)        # the comma makes it a tuple
print(type(single), len(point))
x, y = point         # tuple assignment
print(x, y)
a, b = 1, 2
a, b = b, a          # swap without a temp variable
print(a, b)
first, *rest = (1, 2, 3, 4)
print(first, rest)
```

▶ Run | Reset | 2 ms

```
<class 'tuple'> 2
3 4
2 1
1 [2, 3, 4]
```

Fig. 39 · **Tuple assignment.** Unpacking, swapping and starred assignment.

Tuples as return values

```
def min_max(nums):
    return min(nums), max(nums)   # returns one tuple

low, high = min_max([4, 9, 1, 7])
print(low, high)
q, r = divmod(17, 5)
print(q, r)
print(divmod(17, 5))
```

▶ Run | Reset | 1 ms

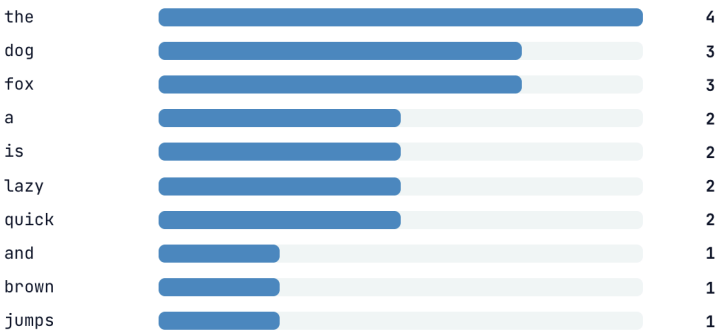
```
1 9
3 2
(3, 2)
```

Fig. 40 · **Tuples as return values.** min_max() and divmod().

Word counter (a dict of counts)

The quick brown fox jumps over the lazy dog. The dog sleeps, the fox runs. A fox is quick and a dog is lazy.

13 different words



`word_count()` uses `counts[w] = counts.get(w, 0) + 1`.

Dictionary methods on {roll: 7, name: Riya, cgpa: 8.9}

- `keys()` -> `['roll', 'name', 'cgpa']`
- `values()` -> `[7, 'Riya', 8.9]`
- `items()` -> `[('roll', 7), ('name', 'Riya'), ('cgpa', 8.9)]`
- `get('zzz', 0)` -> `0`
- `setdefault('new', 1)` -> `{'roll': 7, 'name': 'Riya', 'cgpa': 8.9, 'new': 1}`
- `pop('new')` -> `1, now {'roll': 7, 'name': 'Riya', 'cgpa': 8.9}`

Fig. 41 · **Dictionaries: operations and methods.** A word counter drawn as bars, and keys / values / items / get / setdefault / pop run on a real dict.

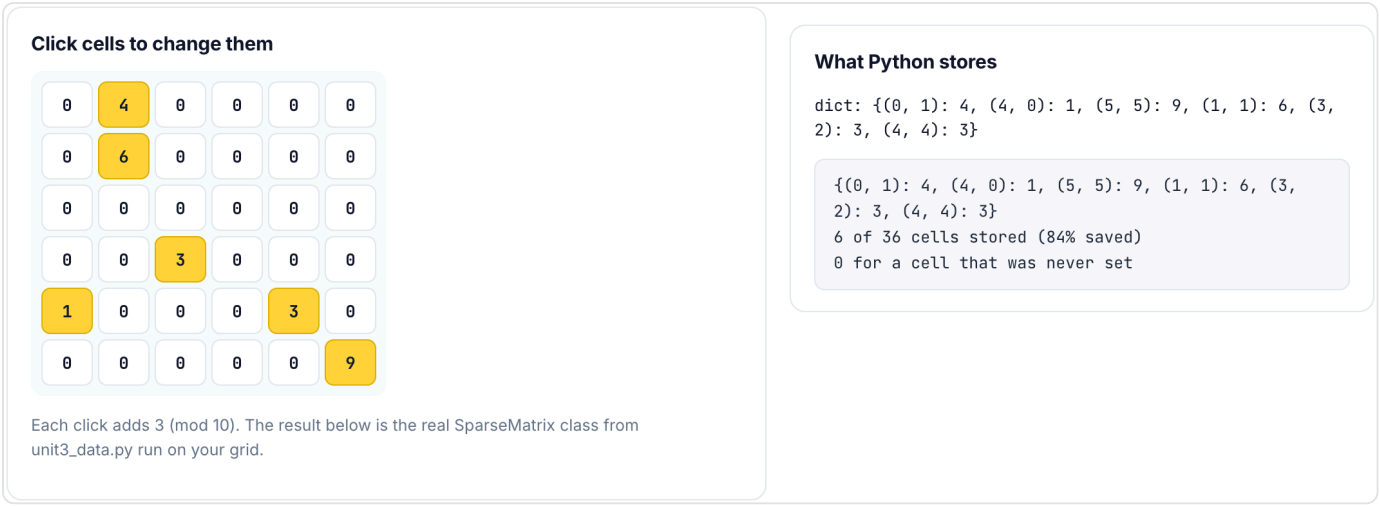


Fig. 42 · **Sparse matrices.** The clicked cells are stored by the real `SparseMatrix` class as a dictionary keyed by (row, col): 6 of 36 cells stored, 84% saved.



Fig. 43 · **Aliasing and copying.** After `alias[0] = "X"` and `a[2][0] = "Y"`: the alias changes with a, the shallow copy shares the inner list, and the deep copy is untouched.



Fig. 44 · **is vs ==.** Equal values, different objects.

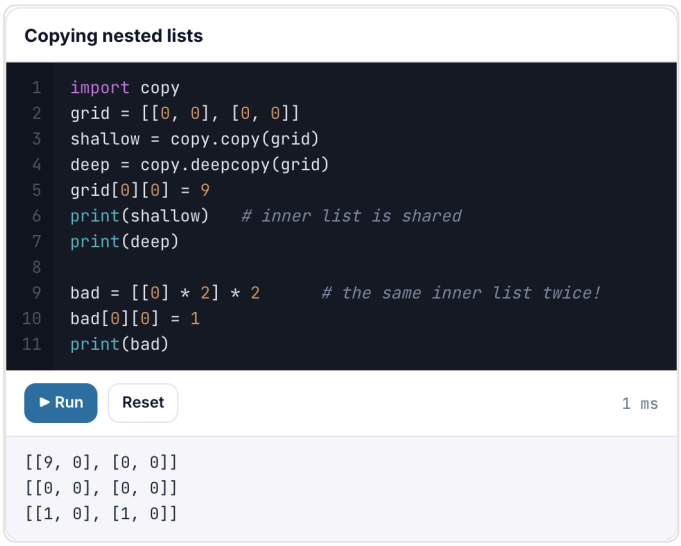


Fig. 45 · **Copying nested lists**, and the `[[0] * 2] * 2` trap.

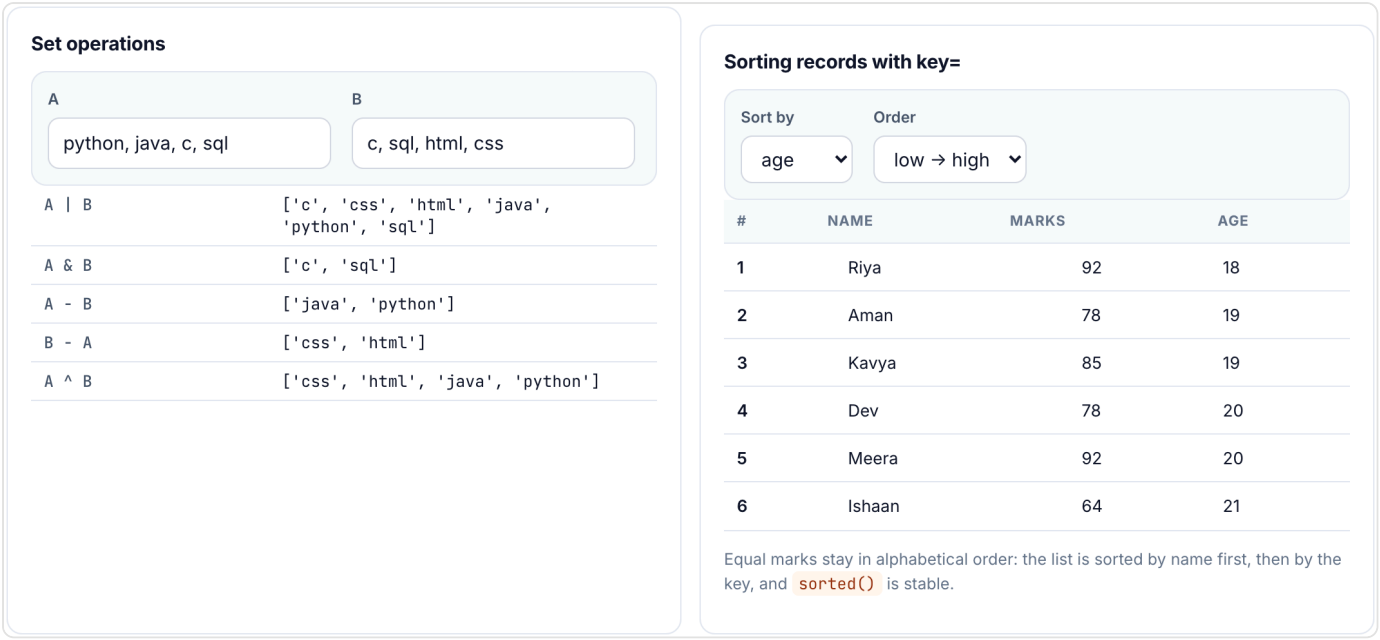


Fig. 46 · **Sets and sorting the data (CO4).** Set operations on two lists of subjects, and records sorted by age from low to high, with ties kept in name order.



Fig. 47 · **Bubble sort pass by pass**, next to sets in code.

07 · UNIT IV

Functions and recursion

Unit IV: function calls, type conversion and coercion, math functions, adding new functions, parameters and arguments, recursion and its use. CO3 Practical 7



Fig. 48 · **Function calls, type conversion and coercion.** "3.75" passed through int(), float(), complex(), bool(), str() and list(); int() refuses it with ValueError. The coercion table shows int + float becoming float.

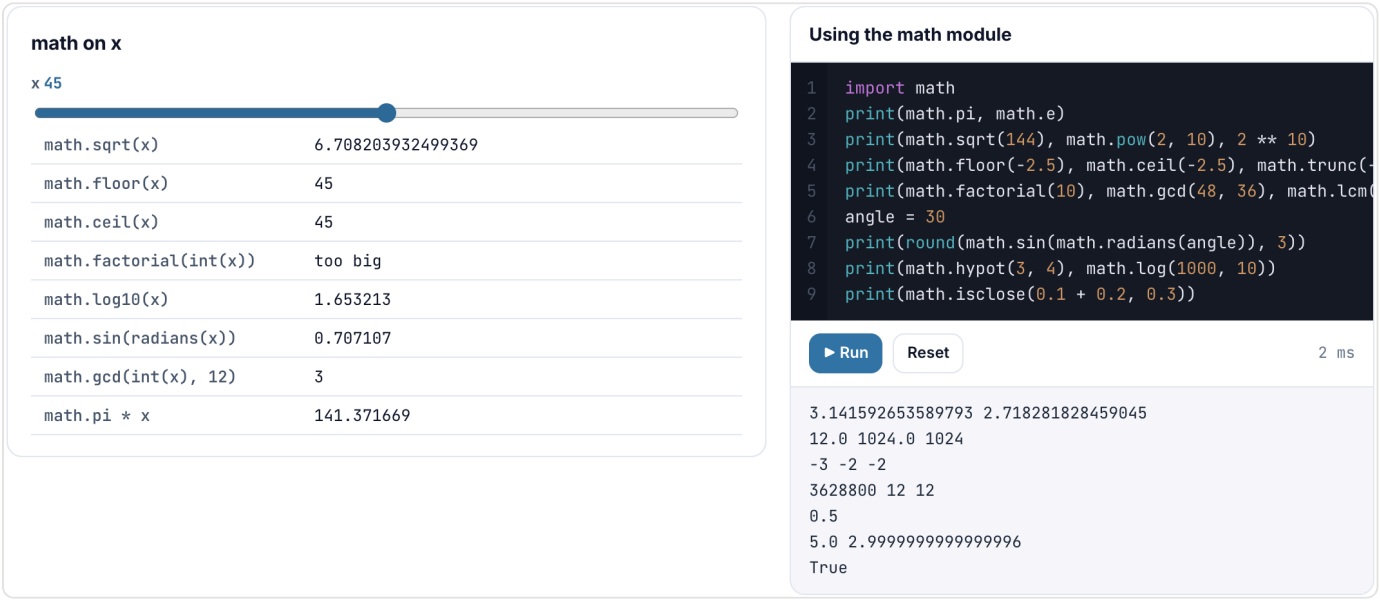


Fig. 49 · **Math functions.** The math module on x = 45, including sin(radians(45)) = 0.707107.

Define and call

```
def greet(name):
    print("Hello,", name)

def square(n):
    return n * n

greet("Riya")
result = square(7)
print(result, square(square(2)))
print(greet("Dev"))    # greet returns None
```

▶ Run | Reset | 1 ms

```
Hello, Riya
49 16
Hello, Dev
None
```

Fig. 50 · **Adding new functions.** def, return, and a function that returns None.

Local and global scope

```
count = 0           # global

def add():
    global count
    count += 1

def local_only():
    count = 100     # a new local variable
    return count

add(); add()
print(count, local_only(), count)
```

▶ Run | Reset | 2 ms

```
2 100 2
```

Fig. 51 · **Local and global scope.**

bill(*items, tax=0.18, discount=0)

Item prices (comma separated): 250, 120, 80

tax % 18

discount ₹: 50

Call: **bill(250, 120, 80, tax=0.18, discount=50)**

Returns: **₹472.0**

Positional / keyword: **3 / 2**

Fig. 52 · **Parameters and arguments.** `bill(*items, tax=, discount=)` called with three positional and two keyword arguments.

Positional, keyword and default

```
def power(base, exp=2):
    return base ** exp

print(power(5))          # exp uses its default
print(power(2, 10))      # positional
print(power(exp=3, base=4))  # keyword, any order

def intro(name, course="INT108", sem=1):
    print(f"{name} | {course} | sem {sem}")

intro("Riya")
intro("Dev", sem=2)
```

▶ Run | Reset | 1 ms

```
25
1024
64
Riya | INT108 | sem 1
Dev | INT108 | sem 2
```

Fig. 53 · **Positional, keyword and default arguments.**

***args and **kwargs**

```
def total(*nums):
    print(type(nums), nums)
    return sum(nums)

print(total(1, 2, 3, 4))

def profile(name, **details):
    print(name)
    for k, v in details.items():
        print(" ", k, "=", v)

profile("Riya", cgpa=8.9, city="Pune")
nums = [5, 6, 7]
print(total(*nums))       # unpack a list into arguments
```

▶ Run | Reset | 1 ms

```
<class 'tuple'> (1, 2, 3, 4)
10
Riya
  cgpa = 8.9
  city = Pune
<class 'tuple'> (5, 6, 7)
18
```

Fig. 54 · ***args and **kwargs**, and unpacking a list into arguments.

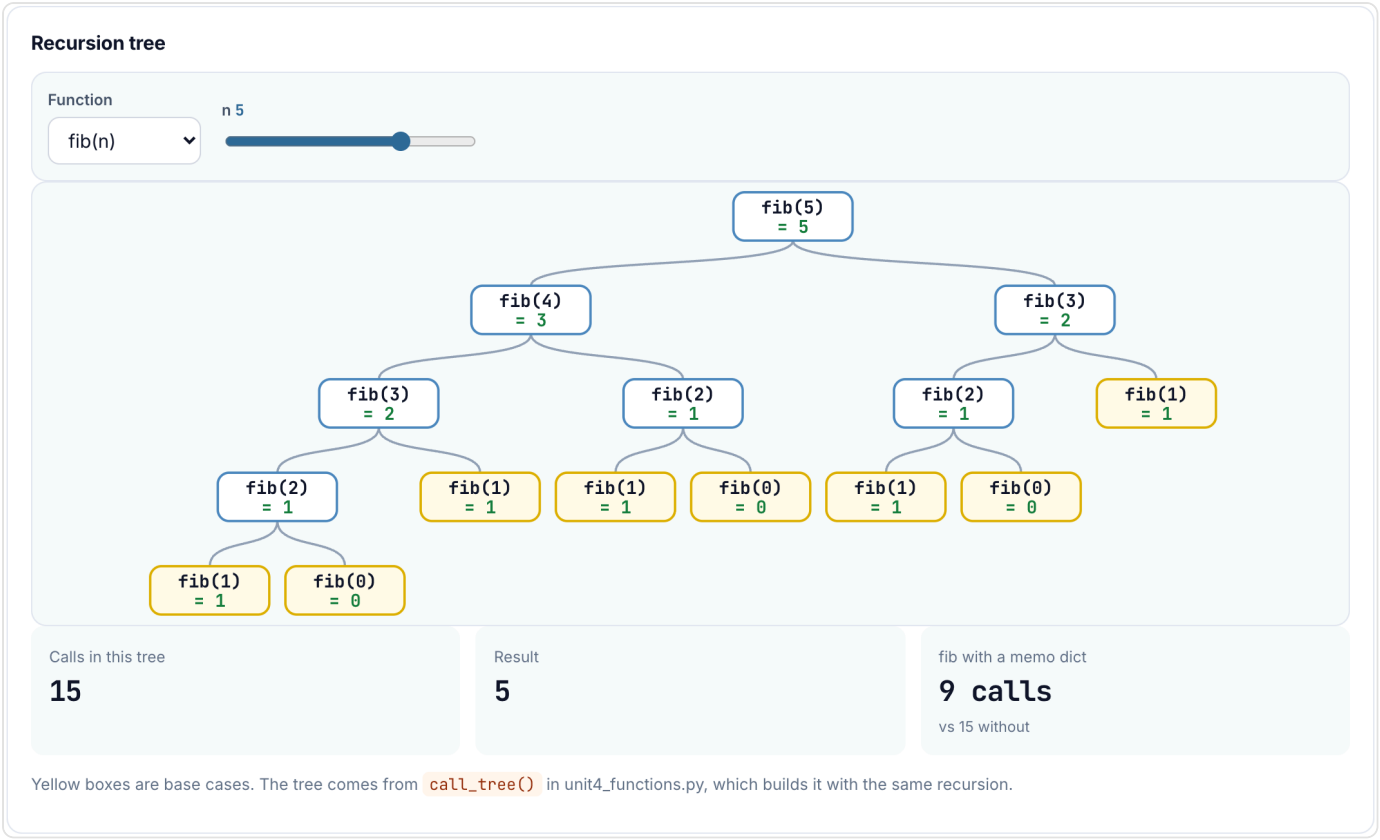


Fig. 55 · **Recursion tree for fib(5).** 15 calls; fib(3) and fib(2) are computed again and again, which is why the memoised version needs only 9.

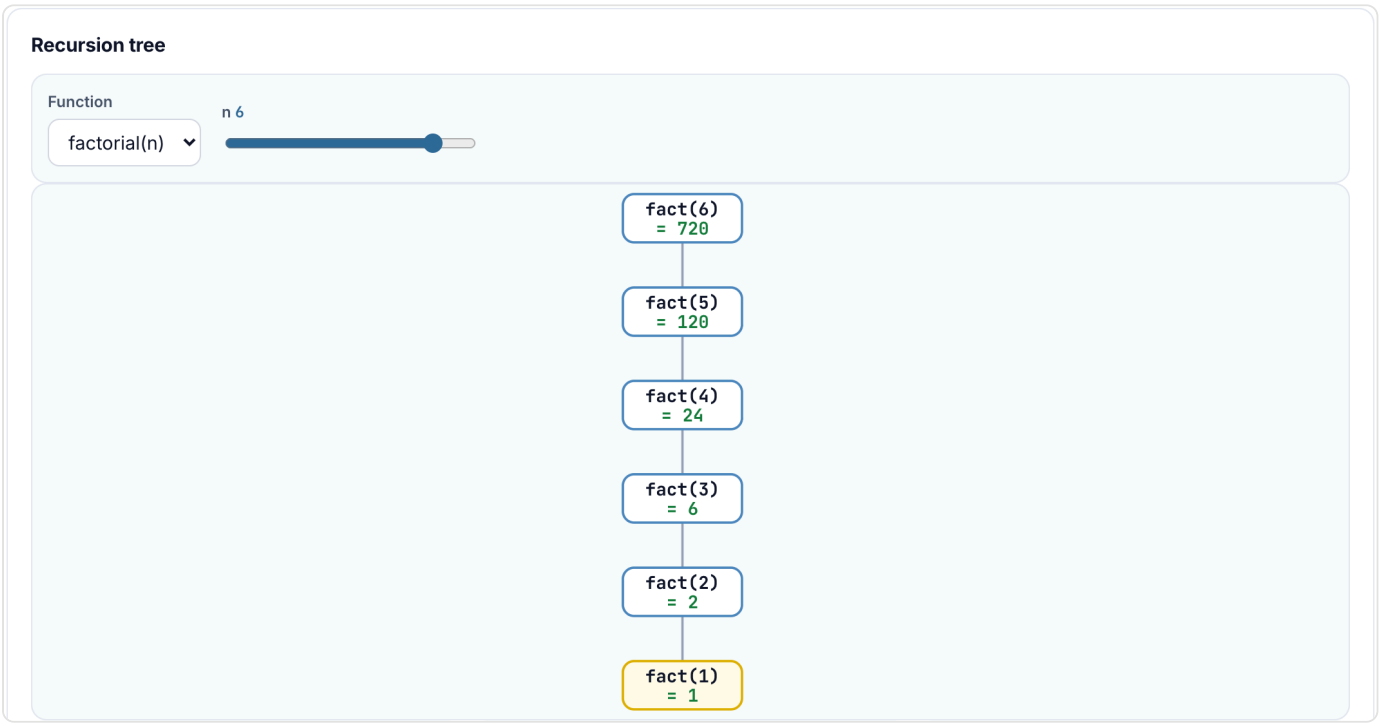


Fig. 56 · **factorial(6).** One call per level, with the results coming back up: 1, 2, 6, 24, 120, 720.

Factorial and Fibonacci

```python
def factorial(n):
    if n <= 1:            # base case
        return 1
    return n * factorial(n - 1)

def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

print(factorial(5), factorial(20))
print([fib(i) for i in range(12)])
```

▶ Run Reset

Press Run (or Ctrl + Enter) to execute this code.

Why fib(n) gets slow (and the fix)

N	PLAIN FIB(N) CALLS	WITH MEMO DICT
5	15	9
10	177	19
15	1973	29
20	21891	39
25	242785	49

Plain fib recomputes the same values again and again; storing them in a dictionary (memoisation) makes it linear.

Fig. 57 · **Why fib(n) gets slow.** Calls with and without a memo dictionary (242,785 against 49 for n = 25), next to the recursive definitions.

Tower of Hanoi: hanoi(n, "A", "B", "C")

Disks 3

7 moves (2^3 - 1)

1. disk 1: A → C
2. disk 2: A → B
3. disk 1: C → B
4. disk 3: A → C
5. disk 1: B → A
6. disk 2: B → C
7. disk 1: A → C

Fast power and digit sum

```python
def power(base, exp):
    if exp == 0:
        return 1
    half = power(base, exp // 2)   # halves the pro
    return half * half * (base if exp % 2 else 1)

def digit_sum(n):
    return n if n < 10 else n % 10 + digit_sum(n //

print(power(2, 30), power(3, 5))
print(digit_sum(98765))
```

▶ Run Reset 3 ms

```
1073741824 243
35
```

Fig. 58 · **Recursion and its use.** Tower of Hanoi with 3 disks in 7 moves ($2^3 - 1$), plus fast power and digit sum.

Recursive binary search

```python
def search(items, target, lo=0, hi=None):
    hi = len(items) - 1 if hi is None else hi
    if lo > hi:
        return -1
    mid = (lo + hi) // 2
    if items[mid] == target:
        return mid
    if items[mid] < target:
        return search(items, target, mid + 1, hi)
    return search(items, target, lo, mid - 1)

nums = [3, 8, 15, 23, 42, 57, 91]
print(search(nums, 42), search(nums, 5))
```

▶ Run Reset 2 ms

```
4 -1
```

Fig. 59 · **Recursive binary search.**

Flatten a nested list

```python
def flatten(items):
    flat = []
    for x in items:
        if isinstance(x, list):
            flat.extend(flatten(x))   # recursive case
        else:
            flat.append(x)            # base case
    return flat

print(flatten([1, [2, [3, 4], 5], [[6]], 7]))
```

▶ Run Reset 2 ms

```
[1, 2, 3, 4, 5, 6, 7]
```

Fig. 60 · **Flattening a nested list.**

08 · UNIT V

Classes, objects and OOP terminology

Unit V: creating classes, creating instance objects, accessing attributes; class inheritance, overriding methods, data hiding and function overloading. Together these give the encapsulation, polymorphism and inheritance of CO5. CO5 Practical 13

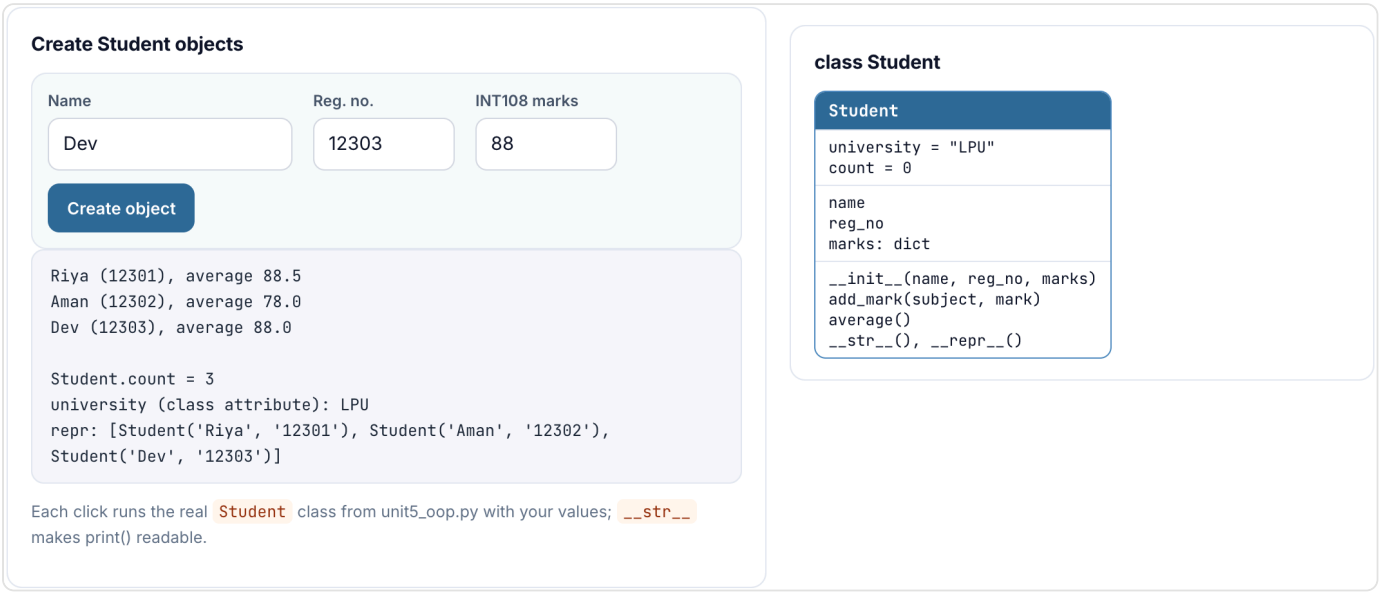


Fig. 61 · **Creating classes and instance objects.** A third Student object ("Dev") was created from the form. `Student.count` is 3 and the class attribute is shared by all three.

Your own class

```
1  class Point:
2      """A point in 2D."""
3
4      def __init__(self, x=0, y=0):
5          self.x = x
6          self.y = y
7
8      def distance_from_origin(self):
9          return (self.x ** 2 + self.y ** 2) ** 0.5
10
11     def __str__(self):
12         return f"({self.x}, {self.y})"
13
14 p = Point(3, 4)
15 q = Point()
16 print(p, q, p.distance_from_origin())
17 print(type(p), isinstance(p, Point))
18 print(p is q, Point.__doc__)
```

▶ Run Reset 6 ms

```
(3, 4) (0, 0) 5.0
<class '__main__.Point'> True
False A point in 2D.
```

Fig. 62 · **A class written from scratch.** __init__ with defaults, a method, __str__, and the docstring.

attribute_report(Student("Riya", "12301", {...}))

type	Student
vars(obj)	{'name': 'Riya', 'reg_no': '12301', 'marks': {'INT108': 92}}
class attributes	{'university': 'LPU', 'count': 4}
methods	['add_mark', 'average']
hasattr(obj, 'name')	True

Reading, adding and deleting attributes

```
class Phone:
    brand = "generic"

    def __init__(self, model):
        self.model = model

p = Phone("X1")
print(p.model, p.brand)
p.price = 15999                # new attribute on 
print(getattr(p, "price"), hasattr(p, "color"))
setattr(p, "color", "black")
print(vars(p))
del p.price
print(getattr(p, "price", "no price"))
print(Phone("Y2").brand, p.color)
print(Phone("Y2").color)   # AttributeError
```

▶ Run | Reset | 9 ms

```
X1 generic
15999 False
{'model': 'X1', 'price': 15999, 'color': 'black'}
no price
generic black
Traceback (most recent call last):
  File "<cell>", line 16, in <module>
AttributeError: 'Phone' object has no attribute 'color'
```

Fig. 63 · **Accessing attributes.** What `vars()`, the class and `hasattr()` see on a Student, and getattr / setattr / del in code.

Class hierarchy in unit5_oop.py

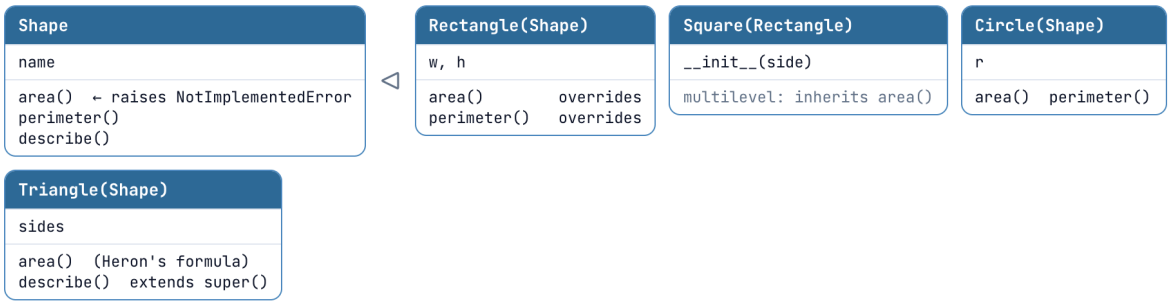


Fig. 64 · **Class inheritance in unit5_oop.py.** Square inherits from Rectangle, which inherits from Shape (multilevel).

Build a shape

Shape: Triangle(a, b, c) | Arguments: 3, 4, 5

```
Triangle: area 6.00, perimeter 12.00, sides (3, 4, 5)
MRO: Triangle -> Shape -> object
overrides area(): True
```

Polymorphism: one loop, many classes

```
from pybench.unit5_oop import Rectangle, Square, Circ

shapes = [Rectangle(3, 4), Square(2), Circle(1.5), Tr
for s in shapes:
    print(s.describe())     # each object runs its ow

biggest = max(shapes, key=lambda s: s.area())
print("biggest:", biggest.name)
print(issubclass(Square, Rectangle), isinstance(shape
```

▶ Run | Reset | 3 ms

```
Rectangle: area 12.00, perimeter 14.00
Square: area 4.00, perimeter 8.00
Circle: area 7.07, perimeter 9.42
Triangle: area 12.00, perimeter 16.00, sides (5, 5, 6)
biggest: Rectangle
True True
```

Fig. 65 · **Overriding methods and polymorphism.** A Triangle(3, 4, 5) built from the form, with its MRO; and one loop calling `describe()` on four different classes, each running its own area().

Write a subclass yourself

```python
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return "..."

    def intro(self):
        return f"{self.name} says {self.speak()}"

class Dog(Animal):
    def speak(self):            # overriding
        return "Woof"

class Puppy(Dog):
    def speak(self):
        return super().speak() + " (tiny)"

for a in (Animal("Generic"), Dog("Bruno"), Puppy("Coco")):
    print(a.intro())
```

▶ Run | Reset | 3 ms

```
Generic says ...
```

Fig. 66 · **super().** Puppy extends Dog's speak() instead of replacing it.

BankAccount("Lukshya", 1000)

Amount ₹ 5000 | deposit | withdraw

```
withdraw(5000) refused: insufficient balance (1500)
balance property: 1500
history: [('deposit', 500)]
vars(acc): {'owner': 'Lukshya', '_history': [('deposit', 500)],
'_BankAccount__balance': 1500}
```

Notice vars(acc) lists the hidden balance as `_BankAccount__balance`.

hiding_demo() output

- `acc.balance -> 1500`
- `acc.__balance -> AttributeError: 'BankAccount' object has no attribute '__balance'`
- `acc._BankAccount__balance -> 1500 (mangled name, still reachable)`
- `acc.balance = 1000000 -> AttributeError: property 'balance' of 'BankAccount' object has no setter`

Fig. 67 · **Data hiding.** A deposit of 500 goes through, then a withdrawal of 5000 is refused. `vars(acc)` shows the hidden balance as `_BankAccount__balance`, and the balance property has no setter.

Try to break in

```python
from pybench.unit5_oop import BankAccount
acc = BankAccount("Riya", 500)
print(acc.balance)
try:
    acc.balance = 1_000_000
except AttributeError as err:
    print("blocked:", err)
acc.__balance = 1_000_000     # creates a NEW attribute, the real one is untouched
print(acc.balance, acc.__balance)
acc.withdraw(800)
```

▶ Run | Reset | 10 ms

```
500
blocked: property 'balance' of 'BankAccount' object has no setter
500 1000000
Traceback (most recent call last):
  File "<cell>", line 10, in <module>
  File "/home/pyodide/app/pybench/unit5_oop.py", line 64, in withdraw
    raise ValueError(f"insufficient balance ({self.__balance})")
ValueError: insufficient balance (500)
```

Fig. 68 · **Trying to break in.** Assigning `acc.__balance` from outside only creates a new attribute; the real balance stays 500.



Fig. 69 · **Operator overloading.** Vector with + − * == abs() len() defined by special methods.

Overloading with default / variable arguments

```
from pybench.unit5_oop import Calculator
c = Calculator()
print(c.add(2, 3))
print(c.add(2, 3, 4, 5))
print(c.add(0.1, 0.2))
print(c.add(0.1, 0.2, rounding=2))

# a second def with the same name replaces the first
def area(r):
    return 3.14 * r * r
def area(l, w):
    return l * w
print(area(3, 4))
print(area(3))   # TypeError: the 1-argument version is gone
```

▶ Run Reset 2 ms

```
5
14
0.30000000000000004
0.3
12
Traceback (most recent call last):
  File "<cell>", line 14, in <module>
TypeError: area() missing 1 required positional argument: 'w'
```

Fig. 70 · **Function overloading in Python:** one add() with *nums and a keyword, and proof that a second def replaces the first.

Your own operator overloading

```
class Money:
    def __init__(self, rupees):
        self.rupees = rupees

    def __add__(self, other):
        return Money(self.rupees + other.rupees)

    def __lt__(self, other):
        return self.rupees < other.rupees

    def __str__(self):
        return f"₹{self.rupees:,}"

wallet = [Money(500), Money(1200), Money(80)]
total = wallet[0] + wallet[1] + wallet[2]
print(total)
print(min(wallet), max(wallet))
print(*sorted(wallet))
```

▶ Run Reset 5 ms

```
₹1,780
₹80 ₹1,200
₹80 ₹500 ₹1,200
```

Fig. 71 · **Your own operators:** Money with __add__, __lt__ and __str__, sorted and summed.

09 · UNIT VI

Files, exceptions and regular expressions

Unit VI: text files, writing variables, reading from and writing to a file, directories, pickling, handling ZeroDivisionError, try–except blocks, the else block and handling FileNotFoundError. Then regular expressions: the concept, various types, the match function and web scraping with regular expressions. CO6 Practicals 8–11, 14, 15

1 · Write a file

```
with open("notes.txt", "w") as f:
    f.write("Unit VI: files and exceptions\n")
    f.write("open() returns a file object\n")
    print("written", f.tell(), "characters")

with open("notes.txt", "a") as f:     # append
    f.write("with closes the file for me\n")

print(open("notes.txt").read())
```

▶ Run Reset 11 ms

```
written 59 characters
Unit VI: files and exceptions
open() returns a file object
with closes the file for me
```

Fig. 72 · **Writing to a file.** "w" then "a", and tell().

2 · Read it three ways

```
with open("notes.txt") as f:
    print(repr(f.read()))           # the whole file

with open("notes.txt") as f:
    print(repr(f.readline()))       # one line
    print(f.readlines())            # the rest, as a list

with open("notes.txt") as f:
    for n, line in enumerate(f, 1): # line by line, memory f
        print(n, line.rstrip())
```

▶ Run Reset 2 ms

```
'Unit VI: files and exceptions\nopen() returns a file
object\nwith closes the file for me\n'
'Unit VI: files and exceptions\n'
['open() returns a file object\n', 'with closes the file for
me\n']
1 Unit VI: files and exceptions
2 open() returns a file object
3 with closes the file for me
```

Fig. 73 · **Reading from a file** with read(), readline(), readlines() and line by line.

Save and load variables (save_variables / load_variables)

```
from pybench.unit6_files import save_variables, load_variables

save_variables("profile.txt", name="Lukshya", marks=92, cgpa=8
print(open("profile.txt").read())

back = load_variables("profile.txt")
print(back)
print(type(back["marks"]), type(back["cgpa"]))
```

▶ Run Reset 51 ms

```
name=Lukshya
marks=92
cgpa=8.7

{'name': 'Lukshya', 'marks': 92, 'cgpa': 8.7}
<class 'int'> <class 'float'>
```

Fig. 74 · **Writing variables.** Numbers stored as text and converted back to int and float.

Make a folder tree (make_dirs / tree)

```
from pybench.unit6_files import make_dirs, tree
import os

make_dirs("project")
with open("project/notes/todo.txt", "w") as f:
    f.write("finish INT108 project\n")
os.rename("project/notes/todo.txt", "project/notes/done.txt")
print("\n".join(tree("project")))
```

▶ Run Reset 8 ms

```
data/
    clean/
    raw/
notes/
    done.txt (22 B)
```

Fig. 75 · **Directories.** makedirs, rename and a folder tree.

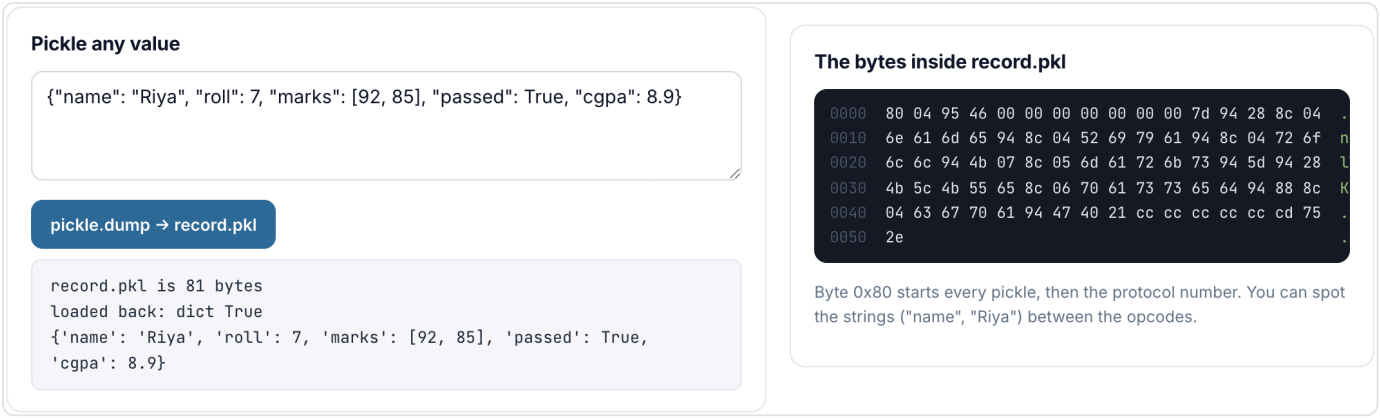


Fig. 76 · **Pickling.** A dict pickled to record.pkl and loaded back equal. The hex dump starts with 0x80 0x04 (protocol 4), and the strings are visible between the opcodes.

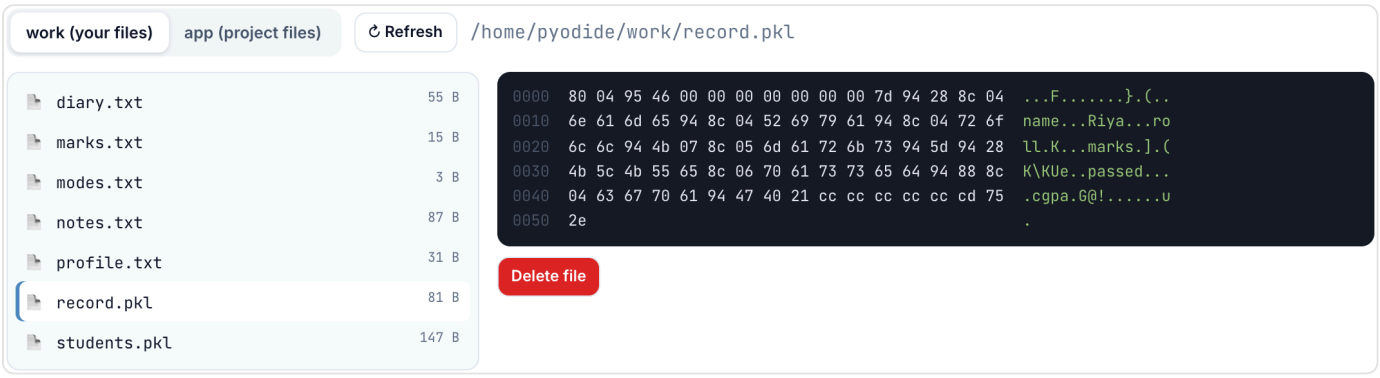


Fig. 77 · **File explorer.** The files created by the cells on the other tabs, with record.pkl opened as a hex dump.

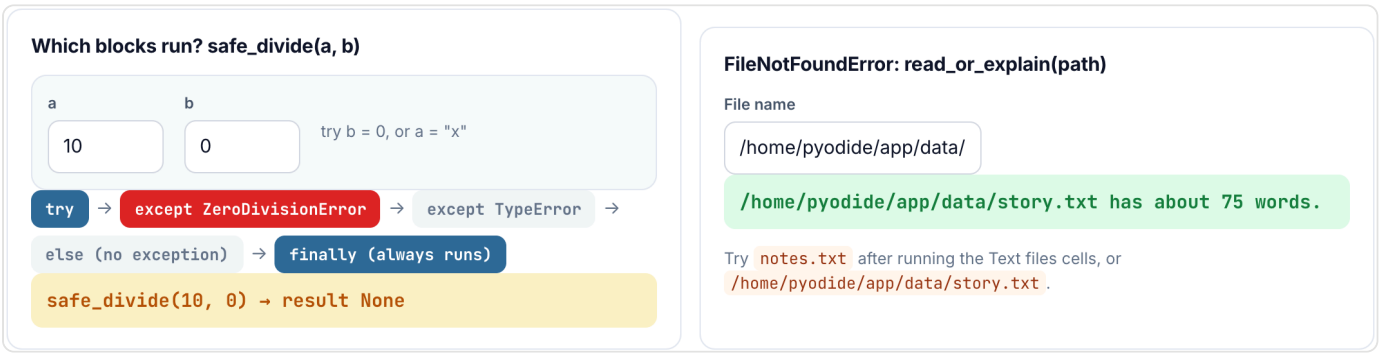


Fig. 78 · **try / except / else / finally.** `safe_divide(10, 0)` goes try → except ZeroDivisionError → finally and skips else. `read_or_explain()` reports the word count of story.txt.

Handling the ZeroDivisionError exception

```
print("Give me two numbers, I will divide them.")
first = input("First number: ")
second = input("Second number: ")
try:
    answer = int(first) / int(second)
except ZeroDivisionError:
    print("You can't divide by 0!")
except ValueError:
    print("Please type whole numbers only.")
else:
    print("Answer:", answer)
finally:
    print("done")
```

Input (one line per input() call)

```
5
0
```

▶ Run Reset 1 ms

```
Give me two numbers, I will divide them.
First number: 5
Second number: 0
You can't divide by 0!
done
```

Fig. 79 · **Handling ZeroDivisionError** with input 5 and 0.

Handling the FileNotFoundError exception

```
filenames = ["notes.txt", "alice.txt", "/home/pyodide/app/data
for name in filenames:
    try:
        with open(name) as f:
            contents = f.read()
    except FileNotFoundError:
        print(f"Sorry, the file {name} does not exist.")
    else:
        print(f"{name} has about {len(contents.split())} words
```

▶ Run Reset 2 ms

```
notes.txt has about 16 words.
Sorry, the file alice.txt does not exist.
/home/pyodide/app/data/story.txt has about 75 words.
```

Fig. 80 · **Handling FileNotFoundError** for several files.

Patterns from unit6_files.PATTERNS (click one)

email `[\w.+-]+@[\w-]+(?:\.[\w-]+)+`	**url** `https?://[^\s\"'<>]+`	**indian mobile** `(?:\+91[- ]?)?[6-9]\d{9}\b`	**date dd-mm-yyyy** `\b(\d{2})[-/](\d{2})[-/](\d{4})\b`	**price in rupees** `(?:Rs\.?|₹)\s?\d[\d,]*(?:\.\d{2})?`
IPv4 address `\b(?:\d{1,3}\.){3}\d{1,3}\b`	**capital words** `\b[A-Z][a-z]+\b`	**LPU reg. no.** `\b1\d{7}\b`		

Fig. 81 · **Regex tester: patterns from** `unit6_files.PATTERNS`.

Pattern and text

Pattern

```
\b(\d{2})[-/](\d{2})[-/](\d{4})\b
```

☐ re.IGNORECASE ☐ re.MULTILINE ☐ re.DOTALL

```
Contact riya.k@lpu.in or aman_23@gmail.com.
Call +91 9876543210 or 8123456789.
Exam on 24-10-2026, results 15/11/2026.
Fee Rs. 1,500 and ₹2,499.00 paid from 192.168.1.10
Reg no 12301234, roll 7
```

Matches

```
Contact riya.k@lpu.in or aman_23@gmail.com.
Call +91 9876543210 or 8123456789.
Exam on 24-10-2026, results 15/11/2026.
Fee Rs. 1,500 and ₹2,499.00 paid from 192.168.1.10
Reg no 12301234, roll 7
```

What re found

matches	2
match() at the start	None
search() first anywhere	24-10-2026
findall()	[['24', '10', '2026'], ['15', '11', '2026']]
groups in pattern	3

#	SPAN	GROUP()	GROUPS()
1	87–97	24-10-2026	['24', '10', '2026']
2	107–117	15/11/2026	['15', '11', '2026']

Fig. 82 · **Regex tester.** The date pattern highlights both dates. match() finds nothing at the start, search() finds the first date and findall() returns the (day, month, year) groups, with each span listed.

First patterns

```
import re
text = "Call 9876543210 or 01822-404404 before 5 pm"
print(re.findall(r"\d", text)[:5])
print(re.findall(r"\d+", text))
print(re.findall(r"\d{10}", text))
print(re.findall(r"\b\w{4}\b", text))
print(re.search(r"\d+ pm", text).group())
```

▶ Run | Reset | 2 ms

```
['9', '8', '7', '6', '5']
['9876543210', '01822', '404404', '5']
['9876543210']
['Call']
5 pm
```

Fig. 83 · **Concept of regular expressions:** first patterns.

Groups and named groups

```
import re
date = re.compile(r"(?P<day>\d{2})-(?P<month>\d{2})-(?P<year>\
m = date.search("Result declared on 15-11-2026")
print(m.groups())
print(m.group("year"), m["month"])
print(m.groupdict())
print(re.findall(r"(\w+)@(\w+)\.in", "a@lpu.in, b@abc.in"))
```

▶ Run | Reset | 2 ms

```
('15', '11', '2026')
2026 11
{'day': '15', 'month': '11', 'year': '2026'}
[('a', 'lpu'), ('b', 'abc')]
```

Fig. 84 · **Types of regular expressions:** groups and named groups.

Form validation with fullmatch()

Check: password | Value: Int108@lpu

✓ valid password: "Int108@lpu"

Rule: at least 8 characters with an uppercase, a lowercase, a digit and a symbol (lookaheads). Uses re.fullmatch().

match vs search vs fullmatch

```
import re
s = "INT108 Python"
print(re.match(r"\d+", s))          # None: s starts
print(re.search(r"\d+", s))         # finds 108
print(re.match(r"[A-Z]+", s).group())
m = re.search(r"(\d)(\d+)", s)
print(m.group(), m.group(1), m.group(2), m.span())
print(re.fullmatch(r"\d{6}", "144411"), re.fullmatch(
```

▶ Run | Reset | 3 ms

```
None
<re.Match object; span=(3, 6), match='108'>
INT
108 1 08 (3, 6)
<re.Match object; span=(0, 6), match='144411'> None
```

Fig. 85 · **Using the match function.** Password validation with lookaheads and `fullmatch()`, next to match vs search vs fullmatch.

Checking user input in a loop

```
import re
while True:
    mobile = input("Mobile number: ")
    if re.fullmatch(r"[6-9]\d{9}", mobile):
        print("saved", mobile)
        break
    print("not a valid 10 digit Indian mobile number, try again")
```

Input (one line per input() call)

```
12345
98765432100
9876543210
```

▶ Run | Reset | 2 ms

```
Mobile number: 12345
not a valid 10 digit Indian mobile number, try again
Mobile number: 98765432100
not a valid 10 digit Indian mobile number, try again
Mobile number: 9876543210
saved 9876543210
```

Fig. 86 · **Checking input in a loop.** Two wrong mobile numbers are rejected before a valid one is saved.

re.sub: search and replace

```
import re
text = "Call 9876543210 or 8123456789"
print(re.sub(r"\d{6}(\d{4})", r"XXXXXX\1", text))   # mask nu
print(re.sub(r"\s+", " ", "too    many   spaces"))
print(re.sub(r"(\w+)@(\w+)", lambda m: m.group(1).upper() + "
print(re.subn(r"a", "4", "banana"))
```

▶ Run Reset 3 ms

```
Call XXXXXX3210 or XXXXXX6789
too many spaces
RIYA at lpu
('b4n4n4', 3)
```

Fig. 87 · **re.sub** masking phone numbers, cleaning spaces, and a function as the replacement.

Formatting

```
name, marks, cgpa = "Riya", 92, 8.765
print(f"{name} scored {marks} with CGPA {cgpa:.2f}")
print(f"{name:<10}|{marks:>5}|")
print("{} scored {}".format(name, marks))
print("%s scored %d" % (name, marks))
print(f"{0.25:.0%}", f"{1234567:,}")
```

▶ Run Reset 2 ms

```
Riya scored 92 with CGPA 8.77
Riya      |   92|
Riya scored 92
Riya scored 92
25% 1,234,567
```

Fig. 88 · **Formatting text** for output (Unit III, used throughout Unit VI).

The page (python/data/shop.html)

Campus Store

Books, stationery and lab supplies near Block 34

Python Crash Course	Fundamentals of Python	Scientific Calculator
₹ 1,299	₹ 899	₹ 1,150
★★★★★	★★★★☆	★★★★★
details	details	details

scrape() result

Page to scrape: sample shop page

title	Campus Store | Books and stationery
headings	Campus Store · Python Crash Course · Fundamentals of Python · Scientific Calculator · Lab Record File · Gel Pens (pack of 10) · Arduino Uno Kit
links (href, text)	/books/python-crash-course (details), /books/fundamentals-of-python (details), /lab/calculator-fx991 (details), /stationery/lab-record (details), /stationery/gel-pens (details), /lab/arduino-kit (details)
prices	₹ 1,299, ₹ 899, ₹ 1,150, ₹ 120, ₹ 150, ₹ 2,499
emails	help@campusstore.in, orders@campusstore.in

Fig. 89 · **Web scraping using regular expressions.** The sample shop page and everything `scrape()` pulls out of its HTML: title, headings, links, prices and e-mails.

scrape_products(): one tuple per product card

Sort products by: rating (high → low)

PRODUCT	PRICE (₹)	RATING
Python Crash Course	1,299	4.8
Arduino Uno Kit	2,499	4.7
Scientific Calculator	1,150	4.6
Fundamentals of Python	899	4.3
Gel Pens (pack of 10)	150	4.1
Lab Record File	120	3.9

The scraping code

```
def scrape(html):
    """Pulls the title, headings, links, prices and email
    with plain regular expressions (fine for simple pages
    title = re.search(r"<title>(.*?)</title>", html, re.S
    return {
        "title": title.group(1).strip() if title else Non
        "headings": [re.sub(r"<[^>]+>", "", h).strip() fo
        "links": re.findall(r'<a[^>]+href="([^"]+)"[^>]*>
        "prices": re.findall(PATTERNS["price in rupees"],
        "emails": sorted(set(re.findall(PATTERNS["email"]
    }


def scrape_products(html):
    """Each product card on the sample shop page becomes
    rx = re.compile(r'<div class="product">.*?<h3>(.*?)</
    return [(name, int(price.replace(",", "")), float(rat
```

Fig. 90 · **Product cards as tuples.** `scrape_products()` returns (name, price, rating), sorted here by rating, next to the scraping code itself.

Every program from the list

Each practical is a file in `python/practicals/` with small functions that can be tested and a `main()` that talks to the user with `input()`. On the site each one has its aim, the concepts it uses, the code in an editor, an input box with sample input, the output and viva questions. Every run is saved to the Lab record, which prints like a lab file.

Practical 3 · Aim: Write a program to check if the entered number is Armstrong or not.

Unit II CO2 Uses: while loop, % 10 and // 10 to take digits, ** power, len(str(n))

practicals/p03_armstrong.py

Input is filled in with a sample, one line per input() call.

```
# Practical 3: check if a number is an Armstrong number
# (sum of each digit raised to the number of digits equals the number, e.g. 153 = 1^3 + 5^3 + 3^3)


def is_armstrong(n):
    digits = len(str(n))
    total = 0
    temp = n
    while temp > 0:
        total += (temp % 10) ** digits
        temp //= 10
    return total == n


def main():
    n = int(input("Enter a number: "))
    k = len(str(n))
    print(" + ".join(f"{d}^{k}" for d in str(n)), "=", sum(int(d) ** k for d in str(n)))
    print(n, "is an Armstrong number" if is_armstrong(n) else "is not an Armstrong number")


if __name__ == "__main__":
    main()

```

Input (one line per input() call)

Fig. 91 · Practical 3 on the site. The aim, the concepts used, the program in the editor, the sample input 153 and the output with the sum of cubes.

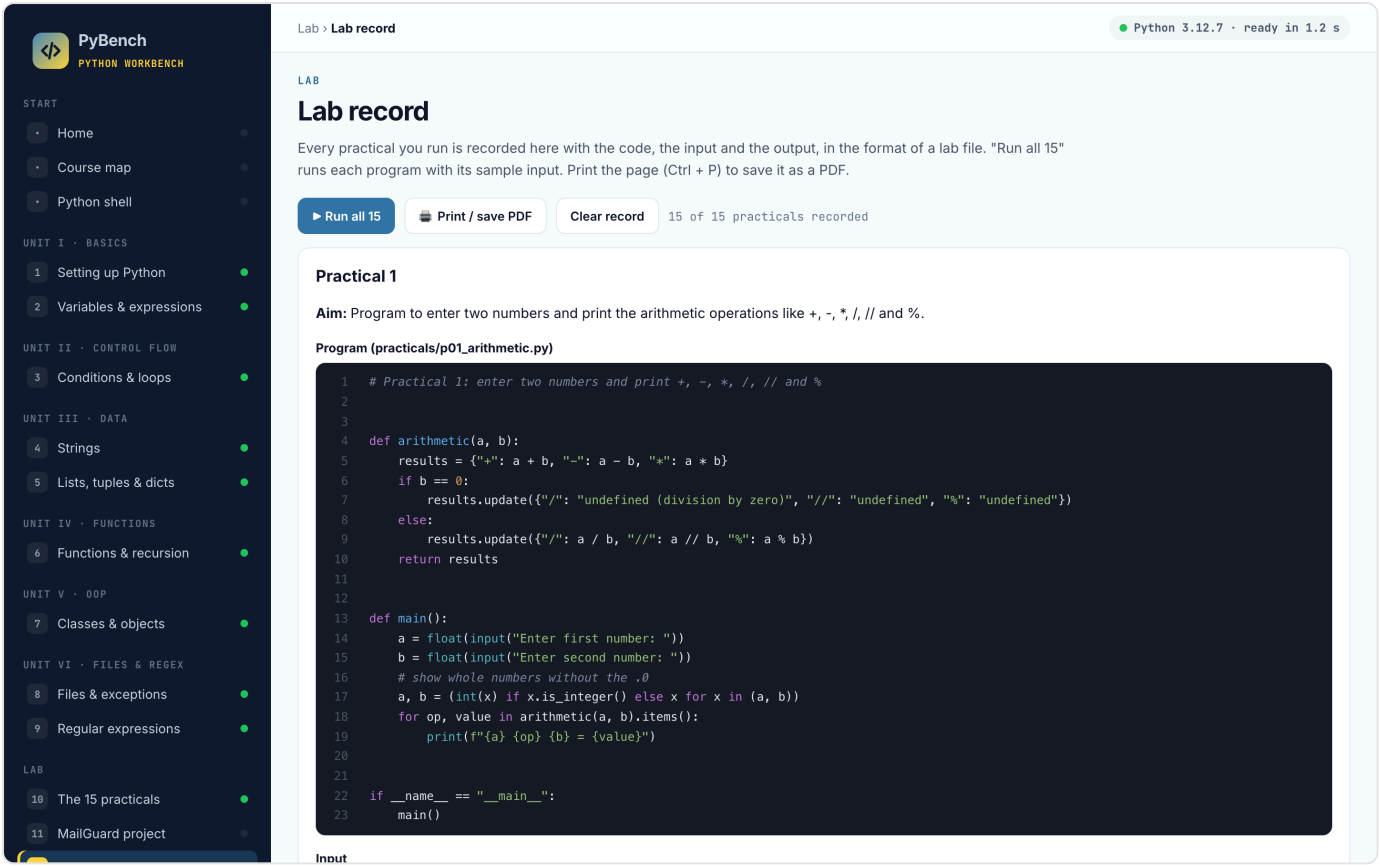


Fig. 92 · **Lab record.** After "Run all 15": all 15 practicals recorded with aim, program, input and output, ready to print.

Runs on my laptop

These outputs come from running each file with `python3` 3.14 on my laptop, with the sample input shown (what is typed after a prompt appears on the same line, as in a terminal).

01 Program to enter two numbers and print the arithmetic operations like +, -, *, /, // and %.

`practicals/p01_arithmetic.py` · Unit I · CO1 · input(), float(), arithmetic operators, dict, f-strings, division by zero check

```
$ python p01_arithmetic.py   (input: 17, 5)
Enter first number: 17
Enter second number: 5
17 + 5 = 22
17 - 5 = 12
17 * 5 = 85
17 / 5 = 3.4
17 // 5 = 3
17 % 5 = 2
```

02 Write a program to find whether an inputted number is perfect or not.

`practicals/p02_perfect.py` · Unit II · CO2 · for loop, %, list of divisors, sum(), if / else

```
$ python p02_perfect.py   (input: 28)
Enter a number: 28
Proper divisors: 1 + 2 + 4 + 7 + 14 = 28
28 is a perfect number
```

03 Write a program to check if the entered number is Armstrong or not.

`practicals/p03_armstrong.py` · Unit II · CO2 · while loop, % 10 and // 10 to take digits, ** power, len(str(n))

```
$ python p03_armstrong.py   (input: 153)
Enter a number: 153
1^3 + 5^3 + 3^3 = 153
153 is an Armstrong number
```

04 Write a program to find factorial of the entered number.

`practicals/p04_factorial.py` · Unit II · CO2 · for loop with range(2, n + 1), accumulator, raising and catching ValueError

```
$ python p04_factorial.py   (input: 6)
Enter a number: 6
6! = 720
```

05 Write a program to enter the number of terms and to print the Fibonacci series.

`practicals/p05_fibonacci.py` · Unit II · CO2 · tuple assignment a, b = b, a + b, for loop, list, print(*list)

```
$ python p05_fibonacci.py   (input: 12)
How many terms? 12
Fibonacci series: 0 1 1 2 3 5 8 13 21 34 55 89
```

06 Write a program to enter the string and to check if it's palindrome or not using loop.

`practicals/p06_palindrome.py` · Unit III · CO4 · string traversal from both ends with a while loop, str.isalnum(), lower()

```
$ python p06_palindrome.py   (input: Never odd or even)
Enter a string: Never odd or even
"Never odd or even" is a palindrome
```

07 Recursively find the factorial of a natural number.

`practicals/p07_recursive_factorial.py` · Unit IV · CO3 · recursion with base case n <= 1, a second version that prints each call and return

```
$ python p07_recursive_factorial.py   (input: 5)
Enter a natural number: 5
factorial(5)
  factorial(4)
    factorial(3)
      factorial(2)
        factorial(1)
        returns 1
      returns 2 * factorial(1) = 2
    returns 3 * factorial(2) = 6
  returns 4 * factorial(3) = 24
returns 5 * factorial(4) = 120

5! = 120
```

08 Read a file line by line and print it.

`practicals/p08_read_lines.py` · Unit VI · CO6 · with open(), iterating over the file object, enumerate(), FileNotFoundError

```
$ python p08_read_lines.py
File name (Enter for story.txt):
 1: Python was created by Guido van Rossum and first
released in 1991.
 2: It is named after Monty Python, not after the
snake.
 3: Readability counts, so Python uses indentation
instead of braces.
 4: A program is a sequence of instructions that
tells a computer what to do.
 5: Files let a program remember data after it stops
running.
 6: Every Python object has a type, a value and an
identity.
 7: Code it, test it, fix it.
 8: Python is fun.
```

09 Remove all the lines that contain the character "a" in a file and write it into another file.

`practicals/p09_remove_a_lines.py` · Unit VI · CO6 · reading and writing three files at once, "w" mode, membership test with in

```
$ python p09_remove_a_lines.py
6 lines contained 'a' and were moved to
lines_with_a.txt
2 lines were kept in without_a.txt

--- without_a.txt ---
Code it, test it, fix it.
Python is fun.

--- lines_with_a.txt ---
Python was created by Guido van Rossum and first
released in 1991.
It is named after Monty Python, not after the snake.
Readability counts, so Python uses indentation
instead of braces.
A program is a sequence of instructions that tells a
computer what to do.
Files let a program remember data after it stops
running.
Every Python object has a type, a value and an
identity.
```

10 Read a text file and display the number of vowels / consonants / uppercase / lowercase characters in the file.

`practicals/p10_char_count.py` · Unit VI · CO6 · isalpha(), isupper(), isdigit(), dictionary of counters

```
$ python p10_char_count.py

File name (Enter for story.txt):
vowels      121
consonants  203
uppercase   14
lowercase   310
digits      4
spaces      67
```

11 Create a binary file with name and roll no. Search for a given roll number and display the name, if not found display appropriate message.

`practicals/p11_binary_roll.py` · Unit VI · CO6 · pickle.dump / pickle.load, "wb" and "rb" modes, dict lookup with get()

```
$ python p11_binary_roll.py   (input: 3, 101, Aman,
102, Riya, 103, Dev, 102)

How many students? 3
Roll no: 101
Name: Aman
Roll no: 102
Name: Riya
Roll no: 103
Name: Dev
Saved 3 records to students.dat

Roll no to search: 102
Roll no 102 belongs to Riya
```

12 Write a random number generator that generates random numbers between 1 and 6 (simulates a dice).

`practicals/p12_dice.py` · Unit II · CO2 · random.randint(1, 6), loop, dict of counts, text bar chart

```
$ python p12_dice.py   (input: 60)

How many times to roll? 60
Rolls: ⚃ ⚃ ⚃ ⚃ ⚀ ⚀ ⚀ ⚁ ⚀ ⚃ ⚃ ⚄ ⚄ ⚀ ⚃ ⚄ ⚃ ⚅ ⚅ ⚀ ⚀ ⚁ ⚃ ⚄ ⚀
⚃ ⚄ ⚃ ⚃ ⚅ ⚀ ⚀ ...

1 ⚀    12  ########
2 ⚁    10  #######
3 ⚂     9  ######
4 ⚃    12  ########
5 ⚄    12  ########
6 ⚅     5  ###
```

13 Write a python program to implement a stack using a list data structure.

`practicals/p13_stack.py` · Unit V · CO4 · CO5 · class Stack wrapping a list, append() / pop(), IndexError on underflow, bracket matching

```
$ python p13_stack.py   (input: push 10, push 20,
push 30, show, pop, peek, check {[()()]}, check ([],
pop, pop, pop, quit)

Commands: push <x>, pop, peek, show, check
<expression>, quit
> push 10
pushed 10 -> [10]  <- top
> push 20
pushed 20 -> [10, 20]  <- top
> push 30
pushed 30 -> [10, 20, 30]  <- top
> show
[10, 20, 30]  <- top (size 3)
> pop
popped 30 -> [10, 20]  <- top
> peek
top is 20
> check {[()()]}
{[()()]} is balanced
> check ([)]
([)] is NOT balanced
> pop
popped 20 -> [10]  <- top
> pop
popped 10 -> []  <- top
> pop
Error: pop from an empty stack (underflow)
> quit
```

14 Take a sample of ten phishing e-mails (any text file) and find most common words.

`practicals/p14_phishing_words.py` · Unit VI · CO4 · CO6 · os.listdir, reading many files, re.findall for words, dict counts, sorting by count, stop words

```
$ python p14_phishing_words.py   (input: 10)

How many top words? 10
Read 10 phishing e-mails

refund         9  #########################
hours          7  ###################
details        6  ################
account        5  #############
pay            5  #############
bank           4  ##########
card           4  ##########
claim          4  ##########
customer       4  ##########
expires        4  ##########
```

15 Read a text file line by line and display each word separated by a #.

`practicals/p15_hash_words.py` · Unit VI · CO6 · reading line by line, str.split(), "#".join()

```
$ python p15_hash_words.py

File name (Enter for story.txt):
Python#was#created#by#Guido#van#Rossum#and#first#releas
It#is#named#after#Monty#Python,#not#after#the#snake.
Readability#counts,#so#Python#uses#indentation#instead#
A#program#is#a#sequence#of#instructions#that#tells#a#co
Files#let#a#program#remember#data#after#it#stops#runnin
Every#Python#object#has#a#type,#a#value#and#an#identity
Code#it,#test#it,#fix#it.
Python#is#fun.
```

11 · MAILGUARD

A phishing mail analyser

Practical 14 asks for the most common words in ten phishing mails. Counting words made it obvious that phishing mails share patterns: they rush you, they ask for an OTP or a card number, the sender's domain doesn't match the brand, and the links hide where they go. MailGuard turns each pattern into a rule class and adds the points up into a score with reasons. `C01–C06`

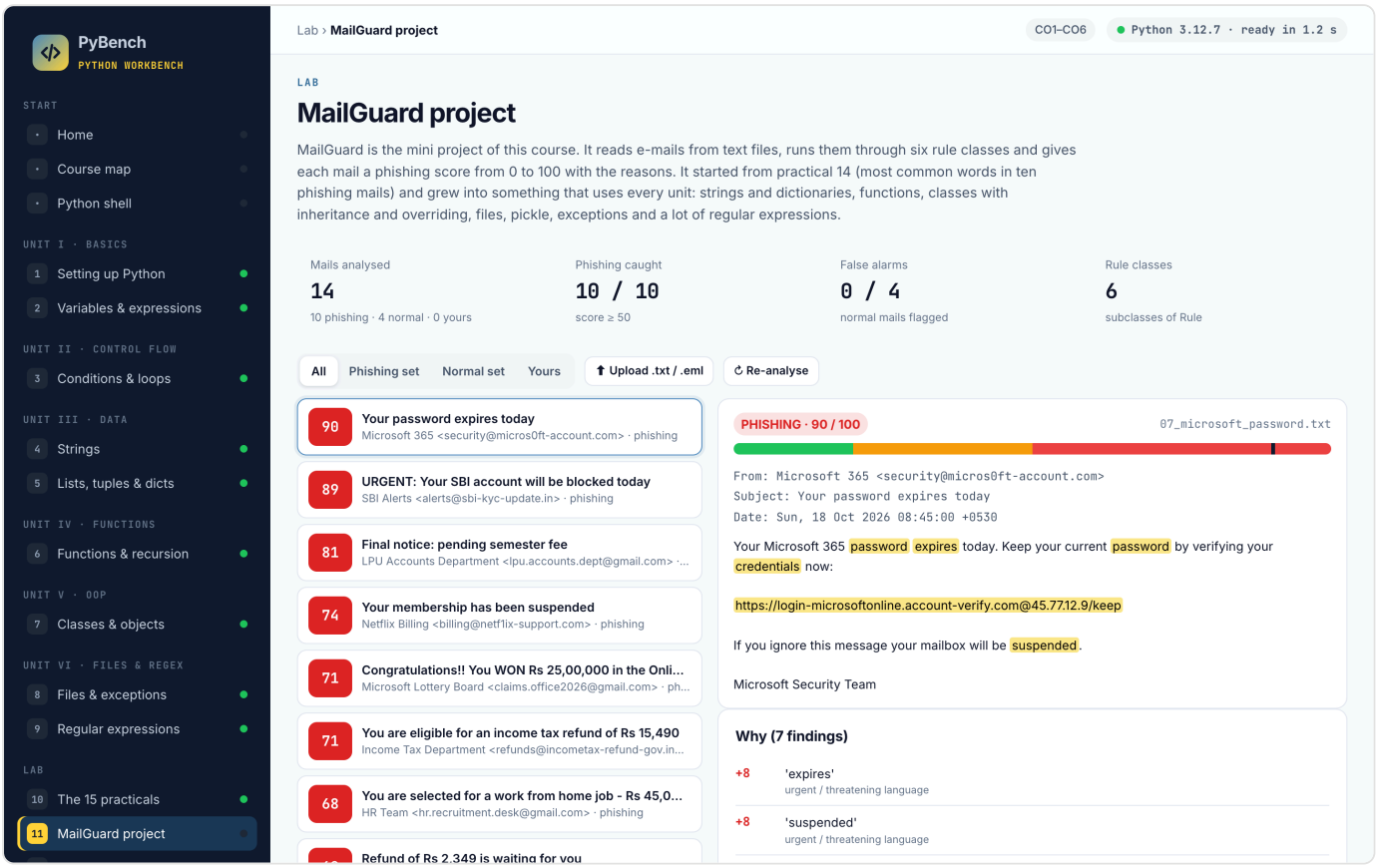


Fig. 93 · **MailGuard.** 14 mails analysed: all 10 phishing mails score 50 or more and the 4 normal mails score 0. The selected mail shows its score gauge, the parts the rules flagged highlighted in the text, and all seven findings with their points.

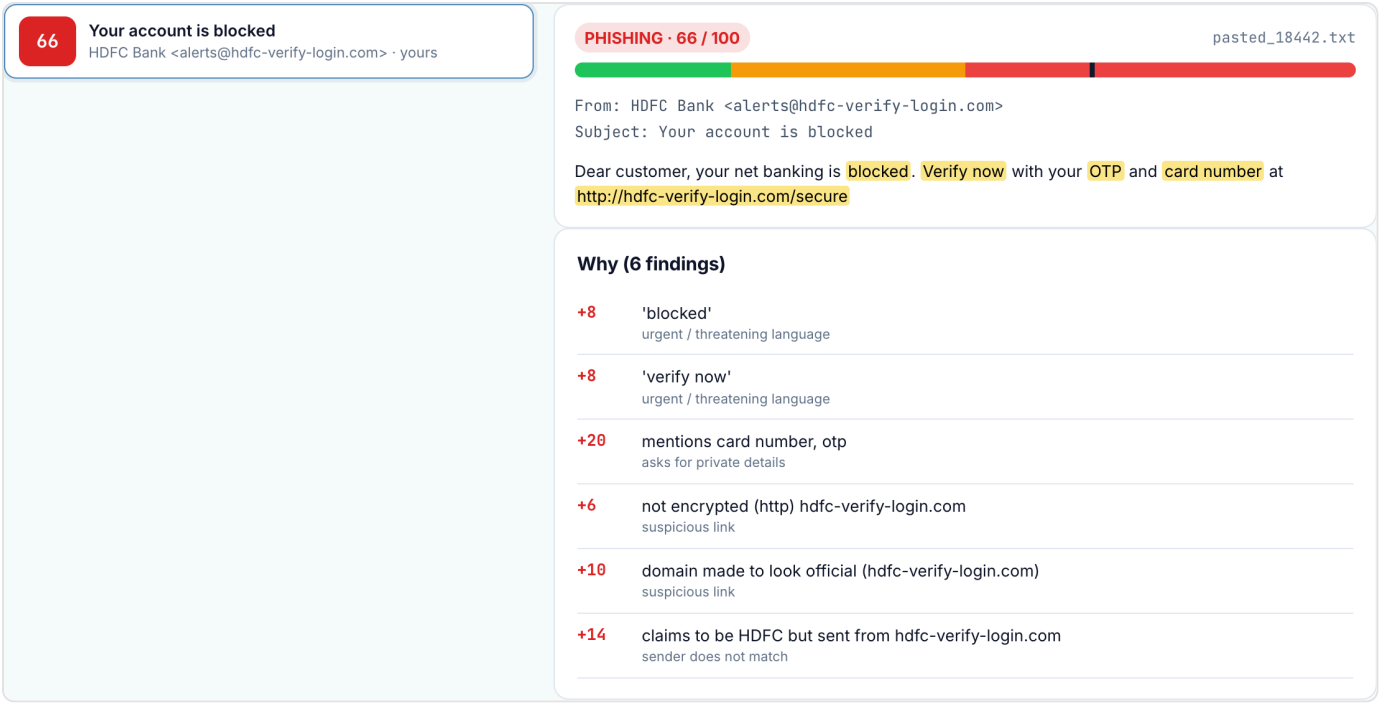


Fig. 94 · **Testing a new mail.** A mail pasted into the page, analysed by the same code: 66, phishing, from six findings: two urgency phrases, the request for OTP and card number, the unencrypted link, a domain made to look official, and a sender claiming to be HDFC from another domain.

How it uses the syllabus

PART	WHAT IT DOES	UNIT
`Mail`	Splits headers from body at the first blank line, reads the headers with a multiline regex, and gives `sender_domain`, `urls` and `words()`; `Mail.from_file()` is a classmethod that reads a file	III, V, VI
`Rule` **+ 6 subclasses**	UrgencyRule, CredentialRule, LinkRule, SenderRule, MoneyRule and AttachmentRule each override `check(mail)`; MailGuard calls `check()` without knowing which rule it has (polymorphism)	V
`Finding`, `Report`	A finding is one reason with points. A report adds them up (max 100), has a `verdict` property and `__lt__`, so a list of reports can be sorted	III, V
`MailGuard`	Keeps its rules in a name-mangled `__rules` list (data hiding). `analyse_folder()` lists a directory and skips unreadable files with try / except	V, VI
`save()` / `load()`	Pickle the reports to a binary file; a missing file gives an empty list instead of a crash	VI
`common_words()`	Counter of words minus stop words, which is practical 14	III

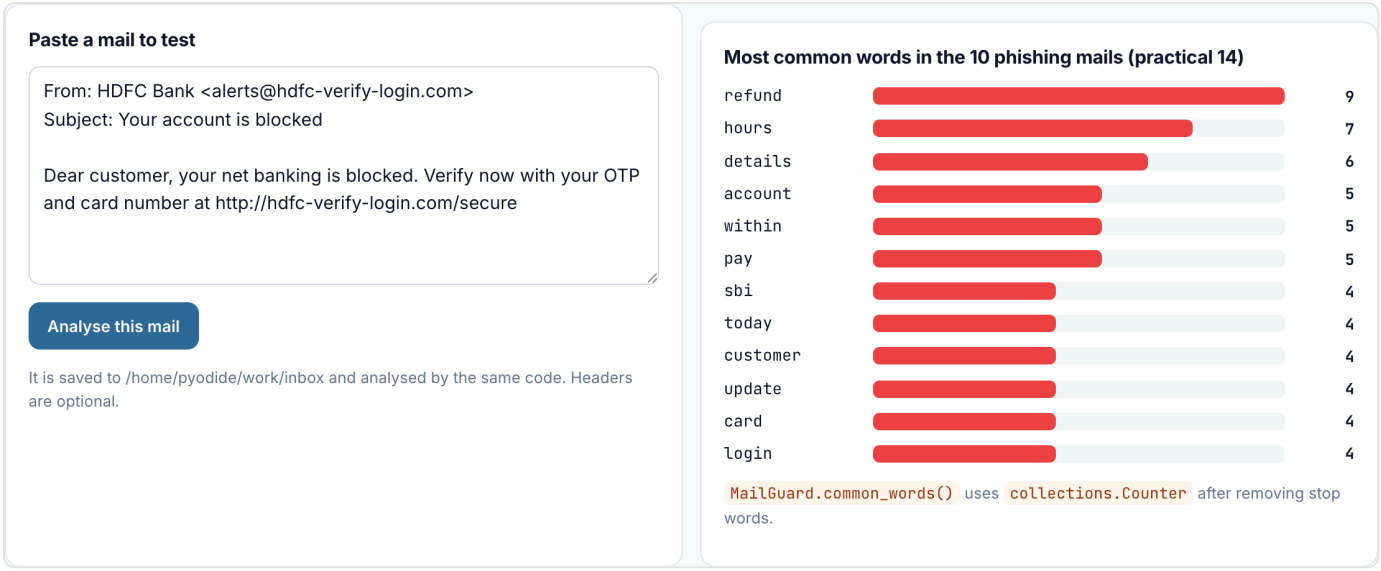


Fig. 95 · **Bottom of the page.** Paste a mail, and the most common words in the phishing set.

Command line

```
$ python -m pybench.mailguard
MailGuard checked 10 mails with 6 rules

 90  phishing    07_microsoft_password.txt Your password expires today
 89  phishing    01_sbi_kyc.txt            URGENT: Your SBI account will be blocked toda
 81  phishing    04_lpu_fee.txt            Final notice: pending semester fee
 74  phishing    03_netflix.txt            Your membership has been suspended
 71  phishing    02_lottery.txt            Congratulations!! You WON Rs 25,00,000 in the
 71  phishing    08_income_tax.txt         You are eligible for an income tax refund of
 68  phishing    09_job_offer.txt          You are selected for a work from home job - R
 60  phishing    05_amazon_refund.txt      Refund of Rs 2,349 is waiting for you
 54  phishing    06_dhl_parcel.txt         Your parcel is on hold - action required
 50  phishing    10_crypto.txt             Double your Bitcoin in 24 hours

Most common words:
  refund       9
  hours        7
  details      6
  account      5
  within       5
  pay          5
  sbi          4
  today        4
  customer     4
  update       4
```

The notebook version

`notebooks/mailguard_analysis.ipynb` runs the same package in a Jupyter / Kaggle notebook. In Kaggle, turn Internet on and the first cell clones the repository. It puts the reports into pandas tables, compares the phishing and normal sets, counts points per rule, extracts every link with a regex, adds a seventh rule as a subclass and saves the reports with pickle. The charts below are the notebook's real output. I executed it with pandas and matplotlib inside Pyodide, because neither is installed on my laptop.



Fig. 96 · **Score of every mail (notebook).** The dashed line is the phishing threshold of 50. The lowest phishing mail scores exactly 50 (crypto) and every normal mail scores 0.

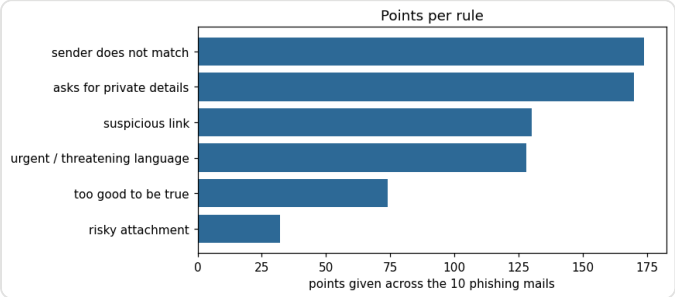


Fig. 97 · **Points per rule.** "Sender does not match" (174) and "asks for private details" (170) do the most work.

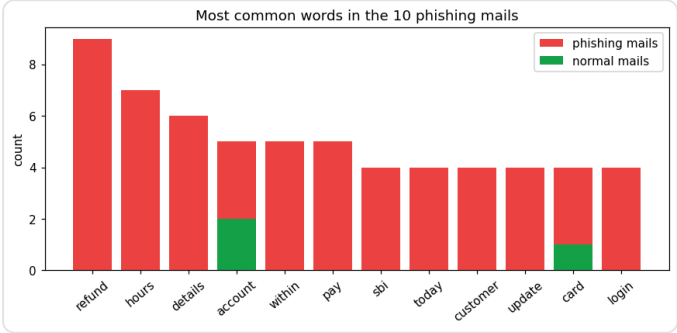


Fig. 98 · **Most common words.** refund, hours, details and pay never appear in the normal mails.

> The sample is small and made by me (10 phishing and 4 normal mails), so the perfect result here does not mean MailGuard would be perfect on real inboxes. The next step would be a larger public dataset and learning the rule weights from it instead of choosing them by hand.

Every topic of the syllabus

This table is generated from the same data as the Course map page (`js/core/syllabus.js`), so the report and the site cannot disagree. "Where" gives the page and tab name.

Course outcomes

CO	STATEMENT	COVERED BY
CO1	describe the installation of python environment and basics of Python language	Setting up Python, Variables & expressions, Python shell, practical 1
CO2	apply the conditional and iterative statements for evaluating the appropriate alternates	Conditions & loops (7 tabs incl. loop tracer), practicals 2–5 and 12
CO3	explore functions, including recursion, with parameters and arguments in Python	Functions & recursion (recursion tree, parameters calculator), practical 7
CO4	construct the core data structures like lists, dictionaries, tuples and sets in Python to store, process and sort the data	Strings, Lists/tuples/dicts/sets/sorting, practicals 6, 13, 14
CO5	apply the concepts of Object-oriented programming as used in Python using encapsulation, polymorphism, and inheritance	Classes & objects (5 tabs), Stack class (practical 13), MailGuard rule classes
CO6	examine file handling operations and effectively apply regular expressions for pattern matching	Files & exceptions, Regular expressions, practicals 8–11, 14, 15, MailGuard

Unit I: Setting up your programming environment; variables, expressions and statements

12 topics

SYLLABUS TOPIC	WHERE IN PYBENCH	WHAT IT DOES
Python versions	`Setting up Python › Python versions`	Timeline 0.9 → 3.14, Python 2 vs 3 table, live sys.version of the running interpreter
Python on Windows	`Setting up Python › Python on Windows`	Install steps, installer screen with "Add python.exe to PATH", Command Prompt checks, troubleshooting table
Running a 'Hello World' program	`Setting up Python › hello`	Interactive vs script mode, runnable Hello World and a first input() program
Naming and using variables	`Variables & expressions › Variables & names`	Live name checker (identifier rules, keywords, built-ins, PEP 8), assignment cell
Avoiding NameError when using variables	`Variables & expressions › Variables & names`	NameError cell to read and fix; name_error_demo() explains the error
Values and types	`Variables & expressions › Values & types`	Type inspector: type, id and mutability of any literal; type() / isinstance() cell
Variable names and keywords	`Variables & expressions › Variables & names`	All keywords listed from keyword.kwlist of the running Python
Statements	`Variables & expressions › Variables & names`	Assignment, chained and multiple assignment statements
Operators and operands	`Variables & expressions › Operators & order`	Operator table with 17 ? 5, arithmetic and augmented assignment cell

SYLLABUS TOPIC	WHERE IN PYBENCH	WHAT IT DOES
Order of operations	`Variables & expressions › Operators & order`	Expression stepper: evaluates any expression one operator at a time using ast (PEMDAS, ** right to left, short-circuit)
Operations on string	`Variables & expressions › String operations`	+ and * on strings with live inputs, str + int TypeError
Composition and comments	`Variables & expressions › Composition & comments`	Composition cell, comment counter (count_comments), docstrings vs comments

Unit II: Conditional and iterative statements `13 topics`

SYLLABUS TOPIC	WHERE IN PYBENCH	WHAT IT DOES
Modulus operator	`Conditions & loops › Modulus operator`	a % b explorer (even, last digit, divisibility, check), clock arithmetic drawing
Random numbers	`Conditions & loops › Random numbers`	Dice simulator with seed and histogram, random module cell
Boolean expressions	`Conditions & loops › Boolean logic`	Truth table generator for any expression in a, b, c
Logic operators	`Conditions & loops › Boolean logic`	and / or / not table, truthiness and short-circuit cell
Conditional	`Conditions & loops › Conditionals`	Grade calculator showing which if / elif / else branch runs
Nested conditionals	`Conditions & loops › Conditionals`	Ticket price (three levels of nesting), leap year, positive/even cell
while statements	`Conditions & loops › while & for`	Collatz while loop; loop tracer steps through while loops
for loop statement	`Conditions & loops › while & for`	range(), strings, step; break / continue / loop else
Nested for	`Conditions & loops › while & for`	Multiplication grid; pattern printer (triangle, pyramid, numbers, letters)
Nested while	`Conditions & loops › while & for`	Prime finder with an inner while
Random numbers in loops	`Conditions & loops › Random numbers`	Monte Carlo π, guess the number game, coin toss loop
Encapsulation and generalization	`Conditions & loops › Encapsulation & generalization`	The multiplication table built in four steps: code → function → parameter → table
Tracing loops (extra)	`Conditions & loops › Loop tracer`	Loop tracer built on sys.settrace: line by line, variables and output

Unit III: Strings, lists, tuples and dictionaries `20 topics`

SYLLABUS TOPIC	WHERE IN PYBENCH	WHAT IT DOES
String: a compound data type	`Strings › Compound type & length`	Index boxes with positive and negative indexes, immutability cell
Length	`Strings › Compound type & length`	len() and the last index, IndexError

SYLLABUS TOPIC	WHERE IN PYBENCH	WHAT IT DOES
String traversal	`Strings › Traversal & slices`	while and for traversal, enumerate
String slices	`Strings › Traversal & slices`	Slice visualizer: any s[start:stop:step] highlighted on the characters
Comparison	`Strings › Comparison`	Explains why one string is smaller using ord() of the first different character
find function	`Strings › find, looping & counting`	Hand-written find() with start position, compared with str.find
Looping and counting	`Strings › find, looping & counting`	count_letter() counter pattern, vowels and words counter
List values, length, membership	`Lists, tuples & dicts › Lists`	list_demo() on your own values: len, in, indexing
List operations, slices	`Lists, tuples & dicts › Lists`	+, *, slices, slice assignment, list methods
Deletion	`Lists, tuples & dicts › Lists`	del by index and slice, remove(), clear()
Accessing elements; list and for loops	`Lists, tuples & dicts › Lists`	for over items and indexes, list comprehensions
List parameters	`Lists, tuples & dicts › Lists`	double_all() changes the caller's list, doubled() returns a new one
Nested list	`Lists, tuples & dicts › Nested lists`	Matrix builder and transpose, marks table
Mutability and tuples	`Lists, tuples & dicts › Tuples`	TypeError when assigning to a tuple item, single-element tuple
Tuple assignment	`Lists, tuples & dicts › Tuples`	Unpacking, swap, starred assignment
Tuple as return values	`Lists, tuples & dicts › Tuples`	min_max_avg() returning three values, divmod()
Dictionaries operations and methods	`Lists, tuples & dicts › Dictionaries`	Word counter with bar chart, keys/values/items/get/setdefault/pop
Sparse matrices	`Lists, tuples & dicts › Sparse matrices`	Clickable grid stored by the SparseMatrix class as {(row, col): value}
Aliasing and copying	`Lists, tuples & dicts › Aliasing & copying`	aliasing_demo(): alias vs shallow vs deep copy, is vs ==
Sets and sorting the data (CO4)	`Lists, tuples & dicts › Sets & sorting`	Set operations, sorting records with key=, bubble sort pass by pass

Unit IV: Functions and recursion `7 topics`

SYLLABUS TOPIC	WHERE IN PYBENCH	WHAT IT DOES
Function calls	`Functions & recursion › Calls & type conversion`	Built-in calls, conversion tester
Type conversion and coercion	`Functions & recursion › Calls & type conversion`	convert() tries int/float/complex/bool/str/list, coercion table

SYLLABUS TOPIC	WHERE IN PYBENCH	WHAT IT DOES
Math functions	`Functions & recursion › Math functions`	math_table(x) with a slider, math module cell
Adding new function	`Functions & recursion › Adding new functions`	def / return, functions calling functions, scope, lambda
Parameters and argument	`Functions & recursion › Parameters & arguments`	bill(*items, tax=, discount=) calculator, defaults, keyword, *args, **kwargs, mutable default trap
Recursion	`Functions & recursion › Recursion`	Recursion tree drawn for fib(n) and factorial(n), call counts with and without memo
Recursion and its use	`Functions & recursion › Uses of recursion`	Tower of Hanoi moves, fast power, digit sum, binary search, flatten

Unit V: Classes and objects; object oriented programming terminology `7 topics`

SYLLABUS TOPIC	WHERE IN PYBENCH	WHAT IT DOES
Creating classes	`Classes & objects › Classes & objects`	Student class with class and instance attributes, class diagram
Creating instance objects	`Classes & objects › Classes & objects`	Create Student objects from a form, __str__ and __repr__
Accessing attributes	`Classes & objects › Accessing attributes`	attribute_report(): vars, class attributes, methods; getattr / setattr / hasattr / del
Class inheritance	`Classes & objects › Inheritance & overriding`	Shape → Rectangle → Square, Circle, Triangle; MRO; build any shape
Overriding methods	`Classes & objects › Inheritance & overriding`	area() and perimeter() overridden, describe() extended with super(), Animal/Dog/Puppy
Data hiding	`Classes & objects › Data hiding`	BankAccount with name-mangled __balance and read-only property, deposit / withdraw
Function overloading	`Classes & objects › Overloading`	Vector operator overloading (+ - * == abs len), Calculator.add(*nums, rounding=)

Unit VI: Files and exceptions; regular expressions `14 topics`

SYLLABUS TOPIC	WHERE IN PYBENCH	WHAT IT DOES
Text files	`Files & exceptions › Text files`	open modes, write / append, file object attributes
Writing variables	`Files & exceptions › Writing variables`	save_variables() / load_variables() and by-hand conversion
Reading from a file	`Files & exceptions › Text files`	read(), readline(), readlines(), line by line
Writing to a file	`Files & exceptions › Text files`	Write what you type with input() into diary.txt
Directories	`Files & exceptions › Directories`	os.getcwd, listdir, makedirs, rename, walk, rmtree; file explorer
Pickling	`Files & exceptions › Pickling`	Pickle any value and see the bytes as a hex dump, pickle Student objects

SYLLABUS TOPIC	WHERE IN PYBENCH	WHAT IT DOES
Handling the ZeroDivisionError exception	`Files & exceptions › Exceptions`	safe_divide() flow showing which of try / except / else / finally run
Using try-except blocks	`Files & exceptions › Exceptions`	Multiple except clauses, failing silently with pass, raise
The else block	`Files & exceptions › Exceptions`	else runs only without an exception (flow highlight and cells)
Handling the FileNotFoundError exception	`Files & exceptions › Exceptions`	read_or_explain(path) and the multiple-files cell
Concept of regular expression	`Regular expressions › Concept & syntax`	Metacharacter table, raw strings, compile
Various types of regular expressions	`Regular expressions › Groups, sub & split`	Groups, named groups, greedy vs lazy, sub, split, flags
Using match function	`Regular expressions › match, search & validation`	match vs search vs fullmatch, form validation, input loop
Web scraping by using regular expressions	`Regular expressions › Web scraping`	scrape() and scrape_products() on a sample shop page and on this site itself

List of practicals

#	AIM	FILE	UNIT · CO
1	Program to enter two numbers and print the arithmetic operations like +, -, *, /, // and %.	`p01_arithmetic.py`	I · CO1
2	Write a program to find whether an inputted number is perfect or not.	`p02_perfect.py`	II · CO2
3	Write a program to check if the entered number is Armstrong or not.	`p03_armstrong.py`	II · CO2
4	Write a program to find factorial of the entered number.	`p04_factorial.py`	II · CO2
5	Write a program to enter the number of terms and to print the Fibonacci series.	`p05_fibonacci.py`	II · CO2
6	Write a program to enter the string and to check if it's palindrome or not using loop.	`p06_palindrome.py`	III · CO4
7	Recursively find the factorial of a natural number.	`p07_recursive_factorial.py`	IV · CO3
8	Read a file line by line and print it.	`p08_read_lines.py`	VI · CO6
9	Remove all the lines that contain the character "a" in a file and write it into another file.	`p09_remove_a_lines.py`	VI · CO6
10	Read a text file and display the number of vowels / consonants / uppercase / lowercase characters in the file.	`p10_char_count.py`	VI · CO6
11	Create a binary file with name and roll no. Search for a given roll number and display the name, if not found display appropriate message.	`p11_binary_roll.py`	VI · CO6
12	Write a random number generator that generates random numbers between 1 and 6 (simulates a dice).	`p12_dice.py`	II · CO2
13	Write a python program to implement a stack using a list data structure.	`p13_stack.py`	V · CO4 · CO5

#	AIM	FILE	UNIT · CO
14	Take a sample of ten phishing e-mails (any text file) and find most common words.	`p14_phishing_words.py`	VI · CO4 · CO6
15	Read a text file line by line and display each word separated by a #.	`p15_hash_words.py`	VI · CO6

How the important parts work

Running a program in the worker

py-worker.js · lines 18–45

```
def _pb_run(code, stdin, filename, as_file):
    buf = io.StringIO()
    lines = list(stdin)

    def fake_input(prompt=''):
        buf.write(str(prompt))
        if not lines:
            raise EOFError('the program asked for more input than you typed in the Input box')
        value = lines.pop(0)
        buf.write(value + '\\n')
        return value

    saved = sys.stdout, sys.stderr, builtins.input
    sys.stdout = sys.stderr = buf
    builtins.input = fake_input
    error = None
    try:
        if as_file:
            runpy.run_path(filename, run_name='__main__')
        else:
            exec(compile(code, filename, 'exec'), {'__name__': '__main__'})
    except SystemExit:
        pass
    except BaseException as err:
        error = _pb_trim(err)
    finally:
        sys.stdout, sys.stderr, builtins.input = saved
    return json.dumps({'out': buf.getvalue(), 'error': error, 'left': len(lines)})
```

Every run goes through this Python function inside the worker. It points `sys.stdout` and `sys.stderr` at one `StringIO`, so prints and errors appear in the order they happened.

`input()` is replaced with `fake_input`, which writes the prompt, takes the next line from the Input box and echoes it, the way a terminal shows what you typed. If the program asks for more input than was given it raises EOFError with a clear message instead of hanging.

Practicals run as files with `runpy.run_path(..., run_name='__main__')`, so their `if __name__ == "__main__"` block runs exactly as with `python p01.py`. Tracebacks are trimmed so they show only your code, not the runner.

Timeouts

py.js · lines 41–52

```
function post(type, payload, timeout) {
    const id = ++seq;
    return new Promise((resolve, reject) => {
        const timer = setTimeout(() => {
            pending.delete(id);
            restart(`stopped after ${timeout / 1000} s, probably an infinite loop`);
            reject(new Error(`TimeoutError: the program was stopped after ${timeout / 1000} seconds (infinite loop?).
Python was restarted.`));
        }, timeout);
        pending.set(id, { resolve, reject, timer });
        worker.postMessage({ id, type, ...payload });
    });
}
```

Python in a worker can't be interrupted unless the server sends special headers, which GitHub Pages doesn't. So each request gets a timer. If it fires, the worker is terminated and a fresh one starts, and the cell shows a TimeoutError. The timer only starts after Python has loaded, so a slow first download is never mistaken for an infinite loop.

The expression stepper

unit1_basics.py · lines 81–126

```
def evaluation_steps(expression):
    """Evaluates an arithmetic expression one operator at a time, in the order
    Python does it (PEMDAS + left-to-right, ** right-to-left). Returns the steps."""
    tree = ast.parse(expression, mode="eval")
    steps = []

    def walk(node):
        if isinstance(node, ast.Constant):
            return node.value
        if isinstance(node, ast.BinOp):
            left = walk(node.left)
            right = walk(node.right)
            result = eval(f"{left!r} {OPS[type(node.op)]} {right!r}")
            steps.append(f"{left!r} {OPS[type(node.op)]} {right!r} = {result!r}")
            return result
        if isinstance(node, ast.UnaryOp):
            operand = walk(node.operand)
            result = eval(f"{OPS[type(node.op)]}{operand!r}")
            steps.append(f"{OPS[type(node.op)]}{operand!r} = {result!r}")
            return result
        if isinstance(node, ast.BoolOp):
            value = walk(node.values[0])
            word = OPS[type(node.op)]
            for nxt in node.values[1:]:
                if (word == "and" and not value) or (word == "or" and value):
                    steps.append(f"{value!r} {word} ... = {value!r} (short-circuit, right side never runs)")
                    return value
                right = walk(nxt)
                result = (value and right) if word == "and" else (value or right)
                steps.append(f"{value!r} {word} {right!r} = {result!r}")
                value = result
            return value
        if isinstance(node, ast.Compare):
            left = walk(node.left)
            result = True
            for op, comp in zip(node.ops, node.comparators):
                right = walk(comp)
                ok = eval(f"{left!r} {OPS[type(op)]} {right!r}")
                steps.append(f"{left!r} {OPS[type(op)]} {right!r} = {ok!r}")
                result = result and ok
                left = right
            return result
        raise ValueError("only numbers, operators and brackets please")

    result = walk(tree.body)
    return steps, result
```

`ast.parse(..., mode="eval")` turns the expression into a tree that already follows Python's precedence rules, so `2 + 3 * 4` becomes Add(2, Mult(3, 4)). Walking the tree depth first and recording each operator as it is evaluated gives the steps in the order Python really uses.

`and` / `or` are handled separately to show short-circuiting: when the left side already decides the answer, the right side is never walked, so `0 and 1 / 0` doesn't raise. Only literals and operators are accepted, so the tool can't be used to run other code.

The loop tracer

unit2_control.py · lines 188–213

```
def trace(source, max_steps=200):
    """Runs a small program and records every line executed with the variables
    at that moment (and what was printed so far). The loop tracer on the site uses it."""
    steps = []
    code = compile(source, "<tracer>", "exec")
    printed = io.StringIO()

    def tracer(frame, event, arg):
        if frame.f_code.co_filename != "<tracer>":
            return tracer
        if event == "line":
            names = {k: v for k, v in frame.f_locals.items() if not k.startswith("__") and isinstance(v, (int, float,
str, bool, list, tuple))}
            steps.append((frame.f_lineno, {k: repr(v) for k, v in names.items()}, printed.getvalue()))
            if len(steps) >= max_steps:
                raise RuntimeError(f"stopped after {max_steps} steps (infinite loop?)")
        return tracer

    scope = {"__builtins__": __builtins__}
    sys.settrace(tracer)
    try:
        with redirect_stdout(printed):
            exec(code, scope)
    finally:
        sys.settrace(None)
    steps.append((None, steps[-1][1] if steps else {}, printed.getvalue()))
    return steps
```

`sys.settrace` makes Python call `tracer` for events in every frame. For each "line" event in the traced program it stores the line number, a copy of the simple local variables and the output printed so far. That is a dry run on paper, done by Python itself.

A step limit stops infinite loops with a clear message, and the tracer is always removed in `finally`. The page then lets you move through the steps and highlights the variable that changed.

The recursion tree

unit4_functions.py · lines 120–137

```
def call_tree(name, n):
    """Records the recursion tree of factorial / fib so the site can draw it.
    Each node is [label, result, children]."""
    def fact_node(k):
        if k <= 1:
            return [f"fact({k})", 1, []]
        child = fact_node(k - 1)
        return [f"fact({k})", k * child[1], [child]]

    def fib_node(k):
        if k < 2:
            return [f"fib({k})", k, []]
        a, b = fib_node(k - 1), fib_node(k - 2)
        return [f"fib({k})", a[1] + b[1], [a, b]]

    if name == "fib":
        return fib_node(min(n, 7))
    return fact_node(min(n, 10))
```

The tree is built by the same recursion it shows: each call returns [label, result, children]. The JavaScript only draws it. Leaves are placed left to right and each parent is centred over its children. n is capped so the fib tree stays readable (fib(7) already has 41 calls).

Sparse matrix and data hiding

unit3_data.py · lines 137–158

```
class SparseMatrix:
    """A matrix that stores only non-zero cells in a dict keyed by (row, col)."""

    def __init__(self, rows, cols):
        self.rows, self.cols = rows, cols
        self.cells = {}

    def __setitem__(self, pos, value):
        if value:
            self.cells[pos] = value
        else:
            self.cells.pop(pos, None)

    def __getitem__(self, pos):
        return self.cells.get(pos, 0)

    def dense(self):
        return [[self[r, c] for c in range(self.cols)] for r in range(self.rows)]

    def saving(self):
        total = self.rows * self.cols
        return f"{len(self.cells)} of {total} cells stored ({100 - 100 * len(self.cells) // total}% saved)"
```

`__setitem__` and `__getitem__` let the class be used like a matrix: `m[2, 3] = 7`. Only non-zero cells are kept in the dictionary; setting a cell to 0 removes it, and `get(pos, 0)` supplies the zeros.

unit5_oop.py · lines 46–66

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self._history = []  # one underscore: "internal, please don't touch"
        self.__balance = balance  # two underscores: name mangled to _BankAccount__balance

    @property
    def balance(self):
        return self.__balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("deposit must be positive")
        self.__balance += amount
        self._history.append(("deposit", amount))

    def withdraw(self, amount):
        if amount > self.__balance:
            raise ValueError(f"insufficient balance ({self.__balance})")
        self.__balance -= amount
        self._history.append(("withdraw", amount))
```

`__balance` is stored as `_BankAccount__balance`, so `acc.__balance` from outside fails. The read-only `balance` property and the two methods are the only way in, and they validate the amount. This is encapsulation.

A MailGuard rule

mailguard.py · lines 100–119

```
class LinkRule(Rule):
    name = "suspicious link"
    weight = 12
    SHORTENERS = ("bit.ly", "tinyurl.com", "t.co", "rb.gy", "cutt.ly", "is.gd")

    def check(self, mail):
        findings = []
        for url in mail.urls:
            host = re.sub(r"^https?://", "", url).split("/")[0].lower()
            if re.fullmatch(PATTERNS["IPv4 address"], host.split(":")[0]):
                findings.append(self.hit(f"link to a bare IP address {host}", 20))
            elif host.endswith(self.SHORTENERS):
                findings.append(self.hit(f"shortened link hides the real site ({host})"))
            elif "@" in url:
                findings.append(self.hit("'@' inside a URL, the part before it is ignored by the browser", 18))
            elif url.startswith("http://"):
                findings.append(self.hit(f"not encrypted (http) {host}", 6))
            if host.count("-") >= 2 or re.search(r"(login|secure|verify|update|account)[\w-]*\.", host):
                findings.append(self.hit(f"domain made to look official ({host})", 10))
        return findings
```

LinkRule overrides `check()`. For each URL found with the shared URL pattern it takes the host and checks, in order: a bare IP address (20 points), a link shortener, an `@` inside the URL (the browser ignores everything before it), or plain http.

Separately it checks for hosts built to look official (`login-`, `secure-`, many hyphens).

Each problem becomes a `Finding` with a readable reason, so the score can always be explained.

mailguard.py · lines 193-236

```
class MailGuard:
    def __init__(self, rules=None):
        self.__rules = list(rules) if rules else [UrgencyRule(), CredentialRule(), LinkRule(), SenderRule(),
MoneyRule(), AttachmentRule()]

    @property
    def rules(self):
        return [r.name for r in self.__rules]

    def analyse(self, mail):
        findings = []
        for rule in self.__rules:
            findings.extend(rule.check(mail))
        return Report(mail, findings)

    def analyse_folder(self, folder):
        reports, errors = [], []
        for name in sorted(os.listdir(folder)):
            if not name.endswith((".txt", ".eml")):
                continue
            try:
                reports.append(self.analyse(Mail.from_file(os.path.join(folder, name))))
            except (OSError, UnicodeDecodeError) as err:
                errors.append(f"{name}: {err}")
        return sorted(reports, reverse=True), errors

    @staticmethod
    def common_words(mails, n=10):
        counter = Counter()
        for mail in mails:
            counter.update(mail.words())
        return counter.most_common(n)

    @staticmethod
    def save(reports, path):
        with open(path, "wb") as f:
            pickle.dump([(r.mail, r.score, r.verdict, [repr(x) for x in r.findings]) for r in reports], f)

    @staticmethod
    def load(path):
        try:
            with open(path, "rb") as f:
                return pickle.load(f)
        except FileNotFoundError:
            return []
```

The rules can be passed in, which is how the notebook and the tests add their own subclass. `analyse_folder()` combines directories, files, exceptions and sorting: reports compare by score through `Report.__lt__`.

Scraping product cards

unit6_files.py · lines 183-186

```
def scrape_products(html):
    """Each product card on the sample shop page becomes a (name, price, rating) tuple."""
    rx = re.compile(r'<div class="product">.*?<h3>(.*?)</h3>.*?₹\s?([\d,]+).*?data-rating="([\d.]+)"', re.S)
    return [(name, int(price.replace(",", "")), float(rating)) for name, price, rating in rx.findall(html)]
```

One compiled pattern with three groups matches a whole product card: the name in `<h3>`, the price after ₹ and the `data-rating` attribute. `.*?` is lazy, so each match stops at the first card's end instead of swallowing the page, and `re.S` lets `.` cross newlines.

The code editor

cell.js · lines 43–78

```
export function editor(code, { onRun, rows } = {}) {
    const pre = h('pre', { class: 'ed__hl', 'aria-hidden': 'true' });
    const area = h('textarea', { class: 'ed__area', spellcheck: false, autocapitalize: 'off', autocomplete: 'off',
'aria-label': 'Python code' });
    area.value = code;
    const gutter = h('div', { class: 'ed__gutter', 'aria-hidden': 'true' });
    const wrap = h('div', { class: 'ed' }, gutter, h('div', { class: 'ed__box' }, pre, area));
    const paint = () => {
        pre.innerHTML = highlight(area.value);
        const n = area.value.split('\n').length;
        gutter.textContent = Array.from({ length: n }, (_, i) => i + 1).join('\n');
        area.style.height = 'auto';
        area.style.height = `${Math.max(area.scrollHeight, (rows ?? 3) * 20 + 20)}px`;
    };
    area.addEventListener('input', paint);
    area.addEventListener('keydown', (e) => {
        if (e.key === 'Enter' && (e.metaKey || e.ctrlKey)) { e.preventDefault(); onRun?.(); return; }
        if (e.key === 'Tab' && !e.shiftKey) {
            e.preventDefault();
            area.setRangeText('    ', area.selectionStart, area.selectionEnd, 'end');
            paint();
        }
        if (e.key === 'Enter') {
            // keep indentation, and indent one more level after a colon
            const before = area.value.slice(0, area.selectionStart);
            const line = before.slice(before.lastIndexOf('\n') + 1);
            let indent = line.match(/^\s*/)[0];
            if (/:\s*(#.*)?$/.test(line)) indent += '    ';
            e.preventDefault();
            area.setRangeText('\n' + indent, area.selectionStart, area.selectionEnd, 'end');
            paint();
        }
    });
    requestAnimationFrame(paint);
    paint();
    return { el: wrap, get value() { return area.value; }, set value(v) { area.value = v; paint(); }, area };
}
```

A plain textarea can't colour text, so the editor puts a highlighted `<pre>` exactly underneath a transparent textarea with the same font and padding. You type into the textarea and see the colours from the layer below, which is repainted on every input.

Enter keeps the indentation and adds a level after a colon, Tab inserts four spaces and Ctrl + Enter runs the cell, which makes it comfortable for Python.

What was checked

Four kinds of tests were run. Unit tests ran on the Python code on my laptop. An automated browser opened every page and tab and pressed every Run button. Full user flows were run in the browser. And the notebook was executed from start to end.

1. Unit tests (python -m unittest, Python 3.14)

TEST CLASS	WHAT IS CHECKED	TESTS	RESULT
Unit1	name rules (keyword, digit, hyphen, built-in), stepper result equals Python's for `2 + 3 * 4 ** 2 // 5`, ** is right to left (512), short-circuit, NameError, literal types	4	**Pass**
Unit2	clock wrap, −7 % 3 = 2, truth table of a and not b, unknown names rejected, grades and ticket prices, leap years 2000 / 1900, Collatz, primes, patterns, guess game, seeded dice repeat, π within 0.05, tracer	6	**Pass**
Unit3	slices and their indices, find / count, comparison, list parameters change the caller (and doubled() doesn't), transpose, swap, min_max_avg, tuple TypeError, word count, sparse matrix removes zeros, alias / shallow / deep, sets, stable sort, bubble sort	6	**Pass**
Unit4	conversion errors, coercion, defaults / keywords / *args, nonlocal scope, factorial with error for −1, fib, fib_memo(80), fast power 3^{13}, Hanoi(5) = 31 moves, digit sum, call counts 21,891 vs fewer than 40	3	**Pass**
Unit5	Student average and validation, attributes, hidden balance not reachable as `__balance`, overdraw refused, areas of four shapes, MRO, abstract area raises, impossible triangle, Vector operators, Calculator.add	4	**Pass**
Unit6	write / append / three reads, variables round trip, directories, pickle round trip starts with 0x80, every exception path, regex emails / dates / mobiles, password and pin validation, scraping a snippet	6	**Pass**
Practicals	perfect numbers below 10,000 are exactly 6, 28, 496, 8128; Armstrong numbers below 1,000; loop and recursive factorial agree; Fibonacci; palindromes; files (missing file message, no "a" left in the kept file); character counts; binary file search found / not found / missing file; dice covers 1–6; stack and brackets; top word "refund"	11	**Pass**
MailGuard	header parsing, all 10 phishing mails caught and sorted, all 4 normal mails pass, IP link and look-alike domain found, custom rule subclass and hidden rules, common words, pickle save / load and missing file	6	**Pass**

```
$ python -m unittest discover -s tests -t .

..........................................
----------------------------------------------------------------------
Ran 46 tests in 0.648s

OK
```

2. Every page and tab in a browser

A script drove Chromium with Playwright. It opened all 15 pages and all 49 tabs, clicked every Run button it could see, and recorded JavaScript errors, console errors and any cell that ended with a Python error. There were no JavaScript or console errors. Twelve cells ended with a Python error, and every one of them is meant to fail as a teaching example: NameError for `Score`, str + int, an index past the end, a missing dictionary key, int("nineteen"),

a missing argument (twice), the RecursionError demo, a missing attribute, an overdraw, a missing folder and an uncaught `raise`.

Looking at the screenshots found real bugs, which I fixed:

- Equal marks were sorted in reverse name order when sorting high to low. Fixed by sorting by name first and relying on `sorted()` being stable.
- The cell input box dropped a trailing empty line, so the "write what you type" program never got its empty line to stop.
- An indent-only line in the shell didn't end a block.
- The font's ligatures drew `==` as one long symbol.
- True / False showed as JSON's true / false.
- The factorial tree was stretched to the full width.
- Bar charts with the default colour were drawn empty, because the colour variable was set to the text "undefined".
- Practical 14 counted header words like "com" and "gmail".
- Every line of the sample text contained an "a", so practical 9 kept nothing.

3. User flows

FLOW	DONE IN THE BROWSER	RESULT
Lab record	"Run all 15" on the Lab record page	**15 of 15 recorded with correct output**
Python shell	assignment, expression echo, a for block and a def block typed with auto-indent, calling the function, 1/0	**42, loop output, 441, ZeroDivisionError traceback**
MailGuard	paste a new mail and analyse it	**scored 66, phishing, with reasons**
Infinite loop	`while True: pass` in a cell	**stopped after 10.1 s, Python restarted, the next cell ran normally**
Phone width	every page at 390 px wide	**no horizontal scrolling on any page**
Notebook	all 10 code cells executed in order (pandas + matplotlib in Pyodide)	**no errors, 3 charts**

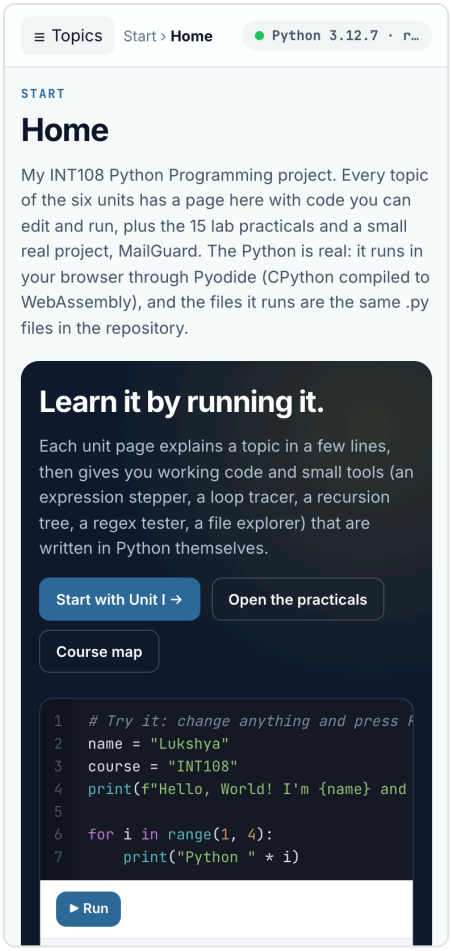


Fig. 99 · Phone: home.

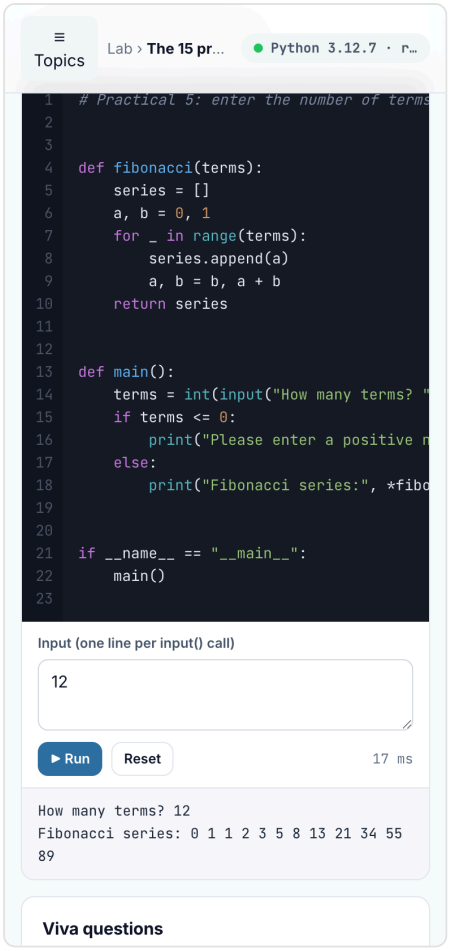


Fig. 100 · Phone: practical 5.

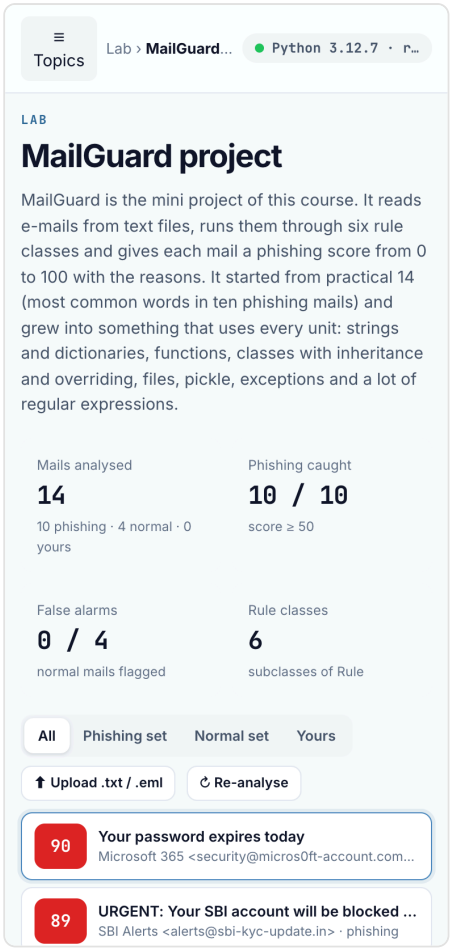


Fig. 101 · Phone: MailGuard.

15 · DEPLOYMENT AND CONCLUSION

Online and what I learned

Deployment

The code is in a public GitHub repository and GitHub Pages serves it straight from the main branch. There is no build step, so the files in the repository are exactly what the browser loads, including the `.py` files that Pyodide runs.

Live site	https://lushadowx.github.io/PyBench/
Source code	https://github.com/LuShadowX/PyBench
Notebook	`notebooks/mailguard_analysis.ipynb` (Kaggle: File → Import notebook, turn Internet on)
Run the site locally	`python -m http.server 8000` in the project folder, then open `http://localhost:8000`
Run the Python locally	`cd python`, then `python -m pybench`, `python practicals/p05_fibonacci.py` or `python -m unittest discover -s tests -t .`

```
$ python -m pybench
PyBench 1.0  |  Python 3.14.7
...
pybench> p2
Enter a number: 496
Proper divisors: 1 + 2 + 4 + 8 + 16 + 31 + 62 + 124 + 248 = 496
496 is a perfect number

pybench> q
```

What I learned

- **Python can explain itself.** The `ast` module shows how an expression is grouped, and `sys.settrace` shows which line runs next. Building the stepper and the tracer taught me the order of operations and loops better than reading about them.
- **Aliasing is easy to get wrong.** `[[0] * 2] * 2` gives the same inner list twice. I hit it myself before I made the aliasing tab.
- **Stable sorting is useful.** Sorting by name first and then by marks keeps ties in alphabetical order, which fixed a real bug in the sorting tool.
- **Inheritance pays off when things get added.** Adding a new MailGuard rule is just a subclass with one method, and nothing else changes. That is what polymorphism is for.
- **Files and exceptions belong together.** Almost every file operation needed a try / except somewhere: a missing file, an unreadable mail, a pickle that isn't there yet.
- **Regex is great for known formats and fragile for everything else.** The scraper works on the sample shop page because I know its layout; a real site would need an HTML parser.

Possible extensions

- Test MailGuard on a public phishing dataset and learn the rule weights from it.
- Save the in-browser files to the browser's storage so they survive a reload.
- Exercises with hidden tests that check the student's own code in a cell.